Institute of Parallel and Distributed Systems

University of Stuttgart
Universitätsstraße 38
D–70569 Stuttgart

Masterarbeit

Connecting MetaConfigurator to the Semantic Web: RDF Authoring, Ontology Integration, and Knowledge Graph Support.

Mahdi Jafarkhani

Course of Study:	Computer Science
Examiner:	Jun. Prof. Dr. rer.nat. Benjamin Uekermann
Supervisor:	Felix Neubauer, M.Sc.

Commenced:	October 15, 2025
Completed:	April 15, 2026

Abstract

Scientific workflows increasingly produce structured JavaScript Object Notation (JSON) data. Although this data is easy to exchange, it is still hard to interpret consistently across systems at the semantic level. JSON Schema helps with structural validation, but it does not provide native Linked Data semantics. This thesis addresses that gap by extending MetaConfigurator, a schema-driven web application for structured data authoring, with Semantic Web capabilities.

The main contribution is an integrated Resource Description Framework (RDF) Authoring Panel that supports end-to-end semantic data workflows in a single user interface. Researchers can start from existing hierarchical or tabular research data, use AI-assisted RDF Mapping Language (RML) to transform it into RDF, and then continue in the same environment to refine triples, run SPARQL queries, visualize knowledge graphs, and export to common RDF serializations.

Overall, the result is a unified workflow that connects traditional non-linked research data with Semantic Web infrastructures. The thesis demonstrates this workflow on a real Metal-Organic Framework (MOF) synthesis application, showing how tabular data can be transformed into RDF, iteratively improved, explored, visualized, and integrated into a target knowledge store.

Contents

List of Figures

List of Tables

List of Algorithms

Abbreviations

Notation	Description	Page List
AI	Artificial Intelligence	30
CEDAR	Center for Expanded Data Annotation and Retrieval	33
ChEBI	Chemical Entities of Biological Interest	62
CHMO	Chemical Methods Ontology	59
CSV	comma-separated values	26
GUI	graphical user interface	28
IRI	Internationalized Resource Identifier	19
JSON	JavaScript Object Notation	15
JSON-LD	JSON for Linked Data	21
JSONPath	JSON Path	26
KG	Knowledge Graph	17
LLM	Large Language Model	30
MOF	Metal-Organic Framework	59
OBI	Ontology for Biomedical Investigations	59
OBO	Open Biological and Biomedical Ontologies	59
OWL	Web Ontology Language	21
QUDT	Quantities, Units, Dimensions, and Data Types	20
R2RML	RDB to RDF Mapping Language	26
RDF	Resource Description Framework	15
RML	RDF Mapping Language	26

Notation	Description	Page List
SHACL	Shapes Constraint Language	21
SPARQL	SPARQL Protocol and RDF Query Language	24
SQL	Structured Query Language	26
W3C	World Wide Web Consortium	21
XML	Extensible Markup Language	15
XPath	XML Path Language	26
YAML	YAML Ain't Markup Language	33

1 Background and Related Work

Computational experiments and simulation workflows now produce large volumes of scientific data [9, 39, 44]. In principle, this data can be modeled and validated directly with Resource Description Framework (RDF)-based standards [10, 40]. In day-to-day practice, however, many researchers still work with spreadsheets or with machine-readable formats such as Extensible Markup Language (XML) and JavaScript Object Notation (JSON) rather than native RDF graphs [25, 27].

The Semantic Web promises better interoperability, explicit semantics, and cross-dataset integration [4, 18, 22], but adopting it remains challenging. Typical hurdles include ontology engineering effort, identifier and vocabulary design, and mapping complexity [18, 22]. To build a practical path through this gap, this chapter starts with JSON and JSON Schema, then introduces core Semantic Web concepts, and finally positions MetaConfigurator as a schema-driven environment for data modeling and editing.

The chapter also reviews related tools for ontology engineering, data transformation, and semantic data management. Many of these tools are strong in their own scope but are commonly used in isolation. As a result, users often have to switch environments to move from structured JSON data to semantically enriched RDF. This thesis addresses that workflow gap by extending MetaConfigurator with semantic transformation, RDF authoring, querying, and visualization in a single user-facing workflow.

1.1 JSON and JSON Schema

This section introduces the data foundations that many scientific workflows already rely on. It first discusses JSON as a lightweight format for structured experimental data, then shows how JSON Schema adds machine-readable structural constraints. Together, they provide a pragmatic baseline for data quality before semantic enrichment and transformation into RDF-based representations.

1.1.1 JSON as a Data Representation Format

JSON is a lightweight data-interchange format widely used for representing structured data in web services, scientific workflows, and configuration systems. JSON encodes data as attribute-

value pairs and ordered lists, making it both human-readable and machine-parseable. Due to its simplicity and broad ecosystem support, JSON has become a de facto standard for structured data exchange across many domains [5].

```json
{
  "parameters": [
    {
      "name": "element_size",
      "value": 0.025,
      "unit": "M"
    }
  ],
  "metrics": [
    {
      "name": "max_von_mises_stress",
      "value": 299791507.5586333,
      "unit": "MegaPA"
    }
  ]
}
```

Listing 1.1: Example JSON representation of simulation input parameters and results

The running example is derived from the benchmark concept presented by Unger et al. for validation and verification of simulation models for concrete structures [37]. The benchmark context is a real finite-element simulation setting in which comparable test cases are executed across tools and mesh resolutions to evaluate reproducibility and implementation consistency. In such a setup, each run must be documented in a machine-readable form so that results can be compared, queried, and reproduced.

Listing 1.1 shows one such run record as two arrays: `parameters` for model inputs and `metrics` for computed outputs. Each entry follows the structure `name`, `value`, and `unit`. In this example, `element_size` is the characteristic mesh length for h-refinement (here $0.025\,\mathrm{m}$, where smaller values mean finer meshes), and `max_von_mises_stress` is the resulting maximum von Mises stress of the simulated specimen (approximately $299.8\,\mathrm{MPa}$) [37].

While JSON provides a convenient serialization format, it does not inherently enforce structural constraints. Therefore, additional mechanisms are required to validate that JSON documents conform to an expected structure.

1.1.2 JSON Schema for Data Validation

JSON Schema offers a standard way to describe the expected structure and constraints of JSON documents. A schema defines properties, data types, required attributes, and validation rules for JSON instances, enabling automated quality checks. The JSON Schema specification formalizes this vocabulary for describing JSON documents and their validation behavior [45].

Listing 1.2 presents a JSON Schema corresponding to the example JSON document in Listing 1.1. The schema mirrors the structure of the instance and ensures that the document contains the expected `parameters` and `metrics` arrays. Each entry in these arrays follows a common structure with the fields `name`, `value`, and `unit`, enforcing that every variable has a textual identifier, a numeric value, and an associated unit.

The schema in Listing 1.2 checks that simulation data has the expected format before it is processed further. It requires every parameter and metric to include a `name`, a numeric `value`, and a `unit`, and it blocks unexpected extra fields (`additionalProperties: false`). This avoids common exchange problems such as missing units, wrong field names, or incorrect data types. At the same time, JSON Schema focuses on structural validity only: it does not encode the domain meaning of terms such as `element_size` or `max_von_mises_stress`, nor does it provide semantic interoperability with Knowledge Graphs (KGs). To address this limitation, semantic mapping techniques are required.

1.2 Semantic Web

The Semantic Web extends the World Wide Web by enabling data to be represented in a machine-readable, semantically rich form. This allows heterogeneous data to be interlinked, interpreted consistently, and queried across systems [4].

1.2.1 Ontology: Modeling Knowledge Domains

Ontologies provide a formal specification of concepts and relationships within a domain of knowledge. One of the most widely cited and influential definitions is provided by Gruber [16]:

> "An ontology is an explicit specification of a shared conceptualization."

In the context of scientific experiments, ontologies can define experiment parameters, measurement units, procedures, and results. For example, an ontology might define a class `PropertyValue` representing any measured property, a class `Method` representing experimental procedures, and relationships such as `hasParameter` and `investigates` to link methods to parameters and observed metrics.

Ontologies serve as the backbone for semantic data integration and reasoning. They allow data from heterogeneous sources to be interpreted consistently, facilitate interoperability, and enable advanced queries across datasets using formal reasoning engines.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Simulation Parameters and Results Dataset",
  "description": "Schema describing input parameters and resulting performance metrics of a simulation run.",
  "type": "object",
  "properties": {
    "parameters": {
      "title": "Simulation Input Parameters",
      "description": "List of input variables.",
      "type": "array",
      "items": {
        "$ref": "#/$defs/variable"
      }
    },
    "metrics": {
      "title": "Simulation Metrics",
      "description": "List of measured or computed results produced by the simulation.",
      "type": "array",
      "items": {
        "$ref": "#/$defs/variable"
      }
    }
  },
  "required": [
    "parameters",
    "metrics"
  ],
  "additionalProperties": false,
  "$defs": {
    "variable": {
      "title": "Quantitative Variable",
      "description": "Represents a numeric quantity with a name and unit, used for both parameters and
          metrics.",
      "type": "object",
      "properties": {
        "name": {
          "title": "Variable Name",
          "type": "string",
          "description": "Identifier of the parameter or metric."
        },
        "value": {
          "title": "Numeric Value",
          "type": "number",
          "description": "Measured or assigned numeric value."
        },
        "unit": {
          "title": "Measurement Unit",
          "type": "string",
          "description": "Unit symbol following a standard QUDT instance."
        }
      },
      "required": [
        "name",
        "value",
        "unit"
      ],
      "additionalProperties": false
    }
  }
}
```

Listing 1.2: JSON Schema validating the simulation result for JSON document in Listing 1.1

1.2.2 RDF: Resource Description Framework

RDF is the core data model of the Semantic Web. It represents information as triples of *subject*, *predicate*, and *object*, which together form a graph structure [10]. This model provides a common, tool-independent way to express entities and relationships across datasets.

Formally, a RDF triple is a 3-tuple [10]:

$$(s, p, o) \in (U \cup B) \times U \times (U \cup B \cup L)$$

where:

- U is the set of all Internationalized Resource Identifiers (IRIs),
- B is the set of blank nodes (anonymous resources),
- L is the set of RDF literals (e.g., strings, numbers, dates),
- $s \in U \cup B$ is the subject,
- $p \in U$ is the predicate (also called property),
- $o \in U \cup B \cup L$ is the object.

A RDF graph G is then defined as a finite set of such triples:

$$G = \{(s_i, p_i, o_i) \mid i = 1, \ldots, n\}, \quad s_i \in U \cup B,\ p_i \in U,\ o_i \in U \cup B \cup L$$

```
<rdf:RDF
 xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
 xmlns:schema="https://schema.org/">
    <schema:PropertyValue rdf:about="https://www.example.org/variable_element_size">
        <schema:name>element_size</schema:name>
        <schema:unitCode rdf:resource="http://qudt.org/vocab/unit/M"/>
        <schema:value rdf:datatype="http://www.w3.org/2001/XMLSchema#double">2.5E-2</schema:value>
    </schema:PropertyValue>
</rdf:RDF>
```

Listing 1.3: RDF/XML representation of parameter `element_size` from Listing 1.1

The RDF model in Listing 1.3 adds an extra layer of meaning to the same data in Listing 1.1. RDF/XML models the data as explicit graph statements instead of nested key-value objects, as depicted in Figures 1.1 and 1.2[1]. In JSON, fields such as `name`, `unit`, and `value` are interpreted mainly by application logic, whereas in RDF the subject (`variable_element_size`) is connected through well-defined predicates (`schema:name`, `schema:unitCode`, `schema:value`) to typed objects, including links to specific ontologies such as Quantities, Units, Dimensions, and Data Types (QUDT). This makes the representation less format-specific and more interoperable, because the meaning is encoded directly in standard IRI and data types rather than implied by local JSON structure.

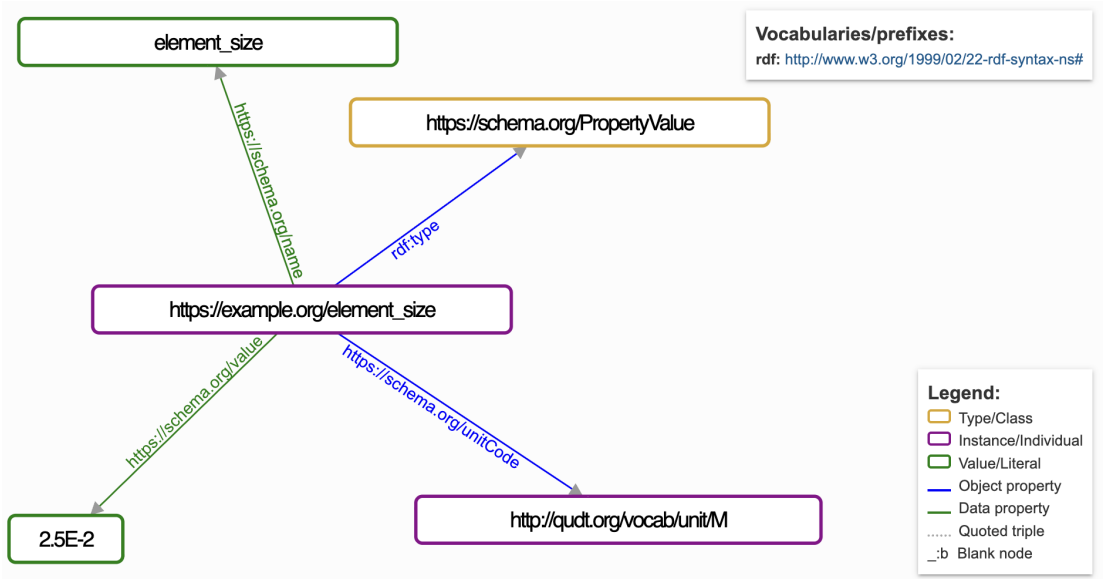

Figure 1.1: Graph view of the RDF representation in Listing 1.3

Subject	Predicate	Object
https://www.example.org/variable_element_size	rdf:type	https://schema.org/PropertyValue
https://www.example.org/variable_element_size	https://schema.org/name	element_size
https://www.example.org/variable_element_size	https://schema.org/unitCode	http://qudt.org/vocab/unit/M
https://www.example.org/variable_element_size	https://schema.org/value	2.5E-2

Figure 1.2: Table view of the RDF representation in Listing 1.3

[1]Visualization and Table view were generated with ISSemantic: `https://issemantic.net/`

1.2.3 JSON-LD as Serialization of RDF

JSON for Linked Data (JSON-LD) is another serialization format for RDF graphs, but expressed in JSON syntax [36]. While RDF/XML (Listing 1.3) uses XML tags, JSON-LD represents the same graph concepts in a JSON structure that is often easier to integrate into existing data pipelines and web-based tools.

A typical JSON-LD document has two key structural components:

- `@context`: This section defines the mapping between JSON keys and ontology IRIs or vocabularies. It provides semantic meaning to each field.
- `@graph`: This array contains the actual data entities, each represented as a node with `@id`, `@type`, and associated properties. Relationships between entities are expressed using references to other `@id` values, forming a connected graph structure.

Listing 1.4 shows a complete JSON-LD representation corresponding to the example in Listing 1.1. The `@context` defines prefixes such as `schema` and `unit`, while the `@graph` contains nodes representing the simulation run, the `element_size` parameter, and the stress metric.

By modeling the experimental data from Listing 1.1 in JSON-LD and explicitly defining its semantics, the data becomes linked to the Semantic Web and can be integrated into a KG. In this representation, `element_size` is modeled as a `schema:PropertyValue` resource with a typed numeric value and an explicit unit IRI, just as in RDF/XML. Compared with Listing 1.3, the semantics are equivalent, but the syntax remains JSON-based.

1.2.4 SHACL: Shapes Constraint Language

The Shapes Constraint Language (SHACL) is a World Wide Web Consortium (W3C) standard for validating RDF graphs against a set of structural and semantic constraints [40]. SHACL performs validation based on explicitly defined "shapes" that describe the expected properties, data types, and cardinality of RDF nodes.

While these shapes can be designed to reflect or align with domain ontologies, SHACL validation itself is independent of ontology languages such as Web Ontology Language (OWL), which are primarily used for logical reasoning rather than constraint checking.

For example, consider the simulation parameter `element_size` in the JSON-LD example (Listing 1.4). Using SHACL, we can define a shape that enforces that every `schema:PropertyValue` representing an experiment parameter must have a numeric `schema:value` and a `schema:unitCode` pointing to a valid unit from the QUDT ontology. By applying this shape, RDF data can be automatically checked for compliance with the expected structure and semantics, preventing errors or inconsistencies before integration into a KG.

```
{
  "@context": {
    "m4i": "http://w3id.org/nfdi4ing/metadata4ing#",
    "schema": "https://schema.org/",
    "unit": "http://qudt.org/vocab/unit/",
    "local": "https://www.example.org/"
  },
  "@graph": [
    {
      "@id": "local:run_fenics_simulation",
      "@type": "m4i:Method",
      "m4i:hasParameter": {
        "@id": "local:variable_element_size"
      },
      "m4i:investigates": {
        "@id": "local:variable_max_von_mises_stress"
      }
    },
    {
      "@id": "local:variable_element_size",
      "@type": "schema:PropertyValue",
      "schema:name": "element_size",
      "schema:unitCode": {
        "@id": "unit:M"
      },
      "schema:value": {
        "@type": "http://www.w3.org/2001/XMLSchema#double",
        "@value": "2.5E-2"
      }
    },
    {
      "@id": "local:variable_max_von_mises_stress",
      "@type": "schema:PropertyValue",
      "schema:name": "max_von_mises_stress",
      "schema:unitCode": {
        "@id": "unit:MegaPA"
      },
      "schema:value": {
        "@type": "http://www.w3.org/2001/XMLSchema#double",
        "@value": "2.997915075586333E8"
      }
    }
  ]
}
```

Listing 1.4: JSON-LD representation of the simulation result presented in Listing 1.1

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix schema: <https://schema.org/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix unit: <http://qudt.org/vocab/unit/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix local: <https://www.example.org/> .

local:PropertyValueShape
    a sh:NodeShape ;
    sh:targetClass schema:PropertyValue ;

    # The variable name label
    sh:property [
        sh:path rdfs:label ;
        sh:datatype xsd:string ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] ;

    # The numeric value of the parameter or metric
    sh:property [
        sh:path schema:value ;
        sh:datatype xsd:double ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] ;

    # The unit, as a named QUDT IRI
    sh:property [
        sh:path schema:unitCode ;
        sh:nodeKind sh:IRI ;
        sh:class qudt:Unit ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] .
```

Listing 1.5: SHACL shape for validating simulation parameters

1.2.5 SPARQL: Querying Semantic Data

SPARQL Protocol and RDF Query Language (SPARQL) is the standard query language for RDF datasets [32], supporting retrieval, filtering, and aggregation of data stored in KGs. Queries are expressed using triple patterns, similar to RDF statements.

Listing 1.6 presents a SPARQL example using the JSON-LD simulation data, where all experiment parameters and metrics are retrieved together with their corresponding values and units.

```
PREFIX schema: <https://schema.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX unit: <http://qudt.org/vocab/unit/>
PREFIX local: <https://www.example.org/>

SELECT ?parameter ?value ?unit
WHERE {
    ?param a schema:PropertyValue ;
           rdfs:label ?parameter ;
           schema:value ?value ;
           schema:unitCode ?unit .
}
```

Listing 1.6: SPARQL query retrieving parameter values and units

This query retrieves the parameters `element_size` and `max_von_mises_stress` together with their corresponding numeric values and associated QUDT units (`unit:M` and `unit:MegaPA`). The result reflects the structure of the JSON-LD representation, in which both parameters and metrics are modeled as `schema:PropertyValue` resources.

This example illustrates how semantic annotations enable structured and consistent access to data. SPARQL further provides mechanisms for querying RDF graphs, including filtering and combining data from multiple sources, which supports the analysis of semantically enriched simulation data.

1.2.6 OWL: Modeling Rich Semantics and Logical Rules

While RDF provides the graph structure and SPARQL enables querying, OWL adds a formal layer for expressing richer domain knowledge [42, 43]. With OWL, one can define class hierarchies, equivalence and disjointness of classes, and logical constraints on properties (e.g., transitive, symmetric, or cardinality-based restrictions). This allows data to be interpreted not only as stored triples, but also through the rules encoded in the ontology. As discussed in Section 1.2.4, SHACL complements this by providing a validation layer for structural and integrity constraints over RDF graphs. Unlike OWL, which focuses on logical inference, SHACL is primarily used to validate whether data conforms to expected shapes, such as required properties, value types, or cardinality restrictions. Together, these technologies support a comprehensive workflow in which RDF represents data, OWL enriches its semantics, SPARQL enables flexible querying, and SHACL ensures data quality and conformance.

Listing 1.7 shows an example of an OWL property definition for `:hasParameter` in the Metadata4Ing ontology [29]. This property is defined as an object property with a domain that is a union of the classes `:Method` and `:Tool`, and a range that points to the class `Variable` from the PIMS-II[2] ontology. The definition also includes a human-readable comment and label.

```
:hasParameter rdf:type owl:ObjectProperty ;
              rdfs:domain [ rdf:type owl:Class ;
                            owl:unionOf ( :Method
                                          :Tool
                                        )
                          ] ;
              rdfs:range <http://www.molmod.info/semantics/pims-ii.ttl#Variable> ;
              rdfs:comment "Points to a parameter of a given method or tool."@en ;
              rdfs:label "has parameter"@en .
```

Listing 1.7: Definition for `:hasParameter` property in Metadata4Ing ontology[29]

An OWL reasoner is a software system that applies ontology axioms to infer implicit facts and check whether a knowledge graph is logically consistent [43]. Using this definition, OWL can derive additional knowledge from very compact assertions. For example, if the graph only specifies the following triple:

```
(local:run_fenics_simulation , m4i:hasParameter , local:variable_element_size)
```

a reasoner infers that

```
(local:variable_element_size, rdf:type, <http://www.molmod.info/semantics/pims-ii.ttl#Variable)
```

[2]The PIMS-II (Process and Instrument Metadata Schema II) ontology is used to model variables and metadata associated with scientific and engineering processes, particularly in simulation and experimental workflows.

from the property range, and that `local:run_fenics_simulation` belongs to the union class `(:Method ⊔ :Tool)` (from the property domain).

This illustrates how OWL models richer semantics than plain triples: the meaning of `:hasParameter` is encoded declaratively in the ontology and reused automatically across all instances. Compared with SHACL, which is mainly used to validate whether data satisfies predefined constraints, OWL focuses on deriving additional knowledge from formally defined semantics. In practice, both are complementary: SHACL helps ensure data quality, while OWL supports richer interpretation and inference in a KG.

1.2.7 RML for Mapping JSON to RDF

RDF Mapping Language (RML) is a declarative language for transforming heterogeneous data sources into RDF representations. RML extends the W3C-standardized RDB to RDF Mapping Language (R2RML) language and supports mappings from JSON, XML, comma-separated values (CSV), and relational databases into RDF datasets [12].

RML mappings are themselves RDF graphs specifying how elements from a source data structure are transformed into RDF triples. Each mapping defines:

- **Logical source:** the data source and reference formulation (e.g., JSON Path (JSONPath), XML Path Language (XPath), Structured Query Language (SQL) query),
- **Subject map:** rules to generate the *subject* of triples,
- **Predicate-object maps:** rules to generate the *predicate* and *object* for triples.

Listing 1.8 shows part of an example RML mapping that converts the JSON simulation output from Listing 1.1 into the JSON-LD structure presented in Listing 1.4.

As one concrete example, `<#ParameterMapping>` iterates over the array `$.parameters[*]` in the JSON document. For each entry, it creates an RDF resource with an IRI constructed from the `name` field (e.g., `local:variable_element_size`) and assigns it the type `schema:PropertyValue`. The JSON attribute `name` is mapped to `rdfs:label`, while the numeric field `value` is mapped to `schema:value`. The unit is converted into a QUDT unit IRI by constructing the resource `http://qudt.org/vocab/unit/{unit}` and assigning it via `schema:unitCode`.

Through this mapping process, structured simulation outputs are automatically converted into RDF triples forming a semantically enriched KG.

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix m4i: <http://w3id.org/nfdi4ing/metadata4ing#> .
@prefix schema: <https://schema.org/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

<#RunFenicsSimulation>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$"
  ] ;

  rr:subjectMap [
    rr:template "https://www.example.org/run_fenics_simulation" ;
    rr:class m4i:Method
  ] ;

  rr:predicateObjectMap [
    rr:predicate m4i:hasParameter ;
    rr:objectMap [ rr:parentTriplesMap <#ParameterMapping> ]
  ] ;

  rr:predicateObjectMap [
    rr:predicate m4i:investigates ;
    rr:objectMap [ rr:parentTriplesMap <#MetricMapping> ]
  ] .

<#ParameterMapping>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$.parameters[*]"
  ] ;

  rr:subjectMap [
    rr:template "https://www.example.org/variable_{name}" ;
    rr:class schema:PropertyValue
  ] ;

  rr:predicateObjectMap [
    rr:predicate rdfs:label ;
    rr:objectMap [ rml:reference "name" ]
  ] ;

  rr:predicateObjectMap [
    rr:predicate schema:value ;
    rr:objectMap [ rml:reference "value" ]
  ] ;

  rr:predicateObjectMap [
    rr:predicate schema:unitCode ;
    rr:objectMap [
      rr:template "http://qudt.org/vocab/unit/{unit}" ;
      rr:termType rr:IRI
    ]
  ] .
...
```

Listing 1.8: Subset of a RML mapping which converts JSON data in Listing 1.1 to JSON-LD in Listing 1.4

1.3 MetaConfigurator for Schema-Driven Data Modeling

The technologies discussed in the previous sections enable structured, validated, and semantically enriched data representations. However, creating and editing such structured data manually can be challenging, particularly for users who are not familiar with textual formats such as JSON or with schema-based validation mechanisms. To address this challenge, tools that support schema-driven data modeling and editing are required.

MetaConfigurator[3] is an open-source web application designed to simplify the creation, editing, and management of structured data files and their accompanying schemas [25, 28]. The tool generates graphical user interfaces (GUIs) dynamically based on a provided schema, allowing users to interact with structured data through intuitive forms rather than manually editing raw text files. By leveraging JSON Schema as the underlying model description, MetaConfigurator enables consistent data validation and guided data entry.

In practice, MetaConfigurator is centered around two main working tabs, **Data** and **Schema**, while **Settings** is used for global configuration [28].

1.3.1 Core Features

- **Visual Schema Editor:** Users can create and modify JSON schemas using a graphical interface, enabling intuitive construction of data models without manually writing schema definitions. Figure 1.4 depicts an example of schema editing in MetaConfigurator, showing the JSON Schema from Listing 1.2. The left panel shows the raw JSON Schema, the middle panel provides a diagram view for editing, and the right panel offers a structured form for schema modification.
- **Simplified and Advanced Schema Modes:** To reduce complexity while preserving expressiveness, MetaConfigurator provides configurable schema-editing modes. A simplified mode hides advanced JSON Schema options, while an advanced mode exposes the full feature set. This behavior is implemented through a meta-schema-builder approach that dynamically derives the editor schema from user settings [28].
- **Schema-Driven Form Generation:** Based on a given JSON Schema, MetaConfigurator automatically generates a graphical form for editing instance data. This ensures that all entered data conforms to the defined structure and constraints.
- **Schema Validation Support:** During data editing, MetaConfigurator validates instance data against the provided JSON Schema, ensuring structural correctness and preventing invalid data entries.

[3] https://www.metaconfigurator.org

- **Schema Visualization and Navigation:** Complex schemas can be explored using a tree-like or diagram-based representation, helping users understand hierarchical structures and relationships between properties. The 2025 extension introduces an interactive diagram-like schema view with synchronized editing actions (e.g., renaming and adding properties), while advanced constructs such as compositions and conditionals are primarily visualized rather than fully edited graphically [28].
- **CSV Import and Schema Inference:** Tabular data can be imported from CSV/Excel-oriented workflows into JSON, with automatic initial schema inference to reduce manual modeling effort [28].
- **Snapshot Sharing and URL-Based Collaboration:** MetaConfigurator supports sharing complete editor states (data, schema, settings) through shareable links and snapshot storage, enabling reproducible collaboration between users [28].

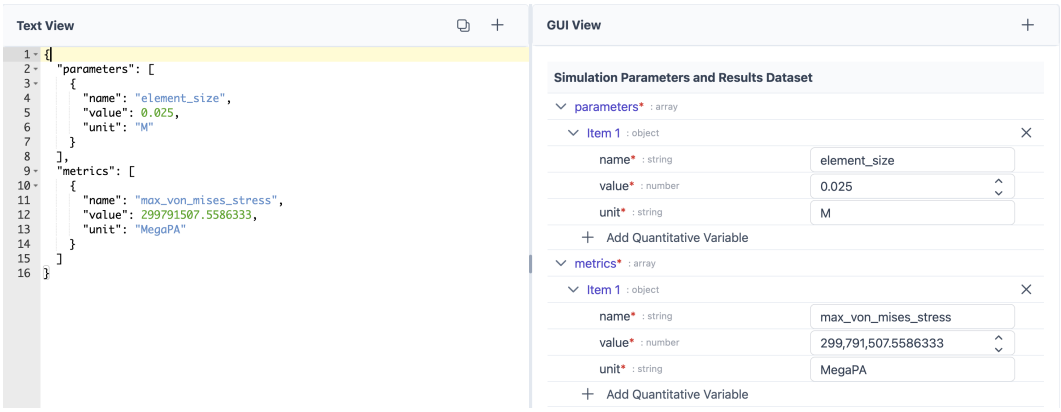

Figure 1.3: A snapshot of MetaConfigurator's data editing interface, showing the JSON instance from Listing 1.1

This combination of views gives users a continuous workflow from low-level text editing to guided schema-driven interaction, and reduces the effort needed to model, validate, and maintain structured data across different domains [25, 28].

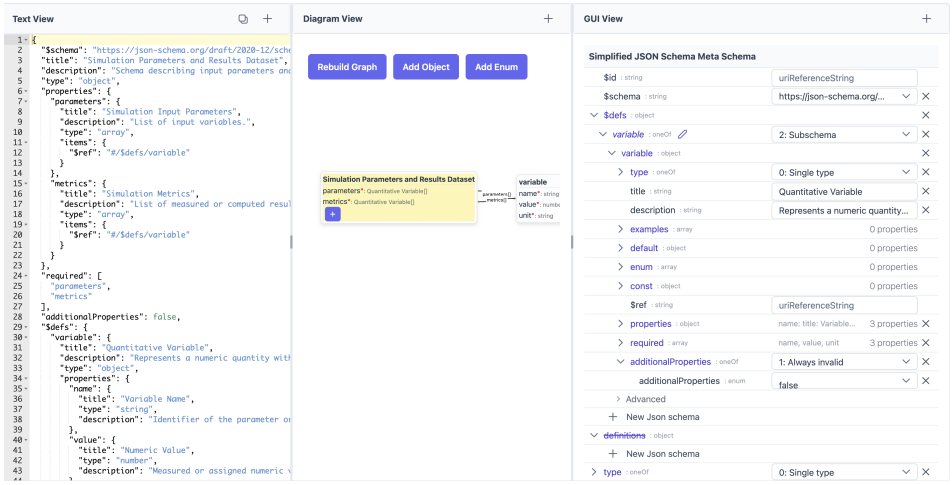

Figure 1.4: A snapshot of MetaConfigurator's schema editing interface, showing the JSON Schema from Listing 1.2

1.3.2 AI-Assisted Schema Engineering and Mapping

Recent work extends MetaConfigurator with Artificial Intelligence (AI)-assisted support for schema creation, modification, and mapping from natural-language user input [26]. The approach combines Large Language Model (LLM)s with deterministic safeguards to improve reliability and usability.

In this workflow, the chat-like dialog is used for data-mapping tasks. Schema edits, in contrast, are applied directly in the schema editor rather than through a separate dialog; users can immediately inspect the result and revert undesired changes with the built-in Undo action. This keeps schema authoring interactive while preserving user control over incremental modifications.

For data transformation, the system generates mapping expressions from user input and executes these mappings deterministically. In the presented implementation, JSONata expressions are generated from an input document and a target schema, then validated and executed in a controlled pipeline. This separates AI-based generation from rule execution and supports scalable transformation workflows across heterogeneous source formats.

1.3.3 Limitations for Semantic Web Integration

Despite its strong support for schema-driven data modeling, MetaConfigurator currently provides only limited support for Semantic Web technologies: while the platform enables the creation and validation of structured data using JSON and JSON Schema, it has limited support for Linked Data representations. Currently, MetaConfigurator treats JSON documents primarily as structured configuration data and does not interpret or expose their underlying RDF semantics.

Several limitations arise from this restriction:

1. **Lack of JSON-LD context authoring:** Users cannot directly author or edit JSON-LD contexts (e.g., the `@context` section that maps JSON keys to ontology IRIs) in the data editing view. Although JSON-LD support allows defining context information in the schema editor, this functionality remains limited and is not available where instance-level data is authored, so linking data elements to ontologies still often requires manual editing outside the core data workflow.
2. **No direct inspection or editing of RDF triples:** MetaConfigurator does not provide mechanisms for inspecting or manipulating RDF triples derived from the underlying JSON data. As a result, users cannot directly view the semantic graph structure implied by JSON-LD documents.
3. **Lack of transformation from JSON to RDF:** The platform lacks built-in mechanisms for transforming conventional JSON documents into semantic representations. As discussed in Section 1.2.7, technologies such as RML enable declarative transformations from JSON to RDF, but such transformations are not currently integrated into MetaConfigurator's workflow.
4. **Lack of semantic querying capabilities:** MetaConfigurator does not support semantic querying of RDF data. As described in Section 1.2.5, SPARQL provides a standard mechanism for querying RDF graphs and retrieving structured information from KGs.
5. **No visualization of RDF KGs:** KGs are often easier to understand when represented visually as nodes and relationships. However, the current platform does not provide mechanisms to visually explore RDF triples or inspect the structure of the resulting KG.

To address these limitations, this thesis extends MetaConfigurator with an **RDF Authoring Panel** that integrates Semantic Web technologies directly into the schema-driven editing environment.

The main contributions of this thesis include:

1. **Integration of RML Mapping Functionality into MetaConfigurator:** The system integrates RML-based mapping functionality into MetaConfigurator to transform conventional JSON documents into JSON-LD. This allows structured JSON data created within the platform to be converted into RDF triples that can be integrated into Semantic Web infrastructures.
2. **RDF authoring and editing support:** The RDF Authoring Panel allows users to directly inspect and edit RDF data. This includes editing the JSON-LD `@context` as well as creating and modifying RDF triples within the interface.
3. **Ontology-aware IRI auto-completion:** The system integrates ontology lookup services that provide auto-completion for IRIs when authoring RDF triples. This facilitates the reuse of existing ontologies and promotes semantic interoperability.
4. **SPARQL querying support:** The extension introduces the ability to query RDF data using SPARQL. This allows users to retrieve and analyze information from the generated KG directly within the MetaConfigurator environment.
5. **Interactive KG visualization:** The RDF triples generated from JSON-LD can be visualized as an interactive graph representation. This visualization enables users to explore entities and relationships within the KG and understand the semantic structure of the data more easily.
6. **AI-assisted SPARQL and RML authoring from user hints:** Building on MetaConfigurator's existing AI-assisted schema capabilities, this thesis introduces AI-assisted generation of SPARQL queries and RML mappings from user-provided hints. Users can describe their intent in natural language and receive draft SPARQL queries and RML mapping definitions that can be reviewed and refined in the RDF Authoring Panel.

By integrating these capabilities into MetaConfigurator, the platform evolves from a schema-based configuration editor into a tool that supports the full workflow of creating, transforming, querying, and exploring semantically enriched data.

1.4 Related Work

Related work in this area spans several mature but often separate tool families: ontology engineering platforms, transformation tools that convert source data into RDF, schema/metadata modeling environments for structured data, and libraries or engines for querying and validation. These families provide strong support for individual tasks, but integrated workflows that combine guided

JSON authoring, semantic transformation, RDF curation, querying, and AI-assisted support in a single user-facing environment are still uncommon.

1.4.1 Ontology Engineering and KG Platforms

Protégé and WebProtégé are widely used for ontology engineering and collaborative ontology curation [7, 19]. Their strength is formal ontology modeling, but this also implies a relatively high entry barrier: users need to understand RDF/OWL concepts and IRIss before they can work productively [18, 22]. In many practical settings, however, data already exists in traditional non-linked formats such as JSON, XML, CSV, or YAML Ain't Markup Language (YAML), and teams do not start from native RDF. MetaConfigurator addresses this gap by supporting import and schema-driven editing of such existing formats [25, 28]; the RDF Authoring Panel could provide the next step as a bridge from conventional structured data to semantic data engineering.

Apache Jena provides a mature programmatic stack for RDF processing, reasoning, and SPARQL querying [13]. GraphDB focuses on RDF storage and query execution in production-oriented KG settings [30]. These systems are strong back-end components, but typically require separate front-end layers for guided data entry and authoring workflows.

1.4.2 Schema and Metadata Modeling for Structured Data

LinkML is a schema-centric tool in this context. It follows a model-driven approach in which a single high-level schema can be used to generate multiple artifacts, including validation schemas and semantic representations such as JSON-LD and OWL [24]. This is highly valuable for maintaining consistency across data models and downstream implementations. At the same time, effective use of LinkML still requires substantial conceptual understanding of schema design, semantic modeling decisions, and artifact generation workflows, which can be challenging for domain users without prior Semantic Web experience. In addition, LinkML introduces yet another schema-language standard that users must learn alongside existing standards such as JSON Schema and RDF, which can increase adoption overhead in practice.

The Center for Expanded Data Annotation and Retrieval (CEDAR) Workbench also improves metadata quality through ontology-assisted authoring in scientific contexts [15]. However, as with other advanced modeling environments, the practical starting point in many projects is not a clean semantic model, but existing non-linked datasets in operational formats such as JSON, XML, CSV, or YAML. For these scenarios, a bridge approach is needed: first support users in editing and structuring their existing data, then map it into RDF. This is why declarative mapping approaches such as RML are particularly suitable in our setting (Section 1.2.7), and why MetaConfigurator is positioned as a gradual path from conventional structured data toward semantic KG workflows rather than assuming that users begin directly with native RDF.

1.4.3 Transformation from Source Data to RDF

Karma, OpenRefine (with RDF extensions), and SPARQL-Generate support transformation from heterogeneous sources to RDF [8, 20, 31]. These tools are strong choices when the main goal is mapping source records into triples. However, they are typically centered on the transformation step itself. In contrast, our approach targets a broader user workflow: preparing existing non-linked data, transforming it to RDF, and then continuing with semantic authoring, inspection, and querying in one environment.

1.4.4 Querying, Validation, and Programmatic RDF Processing

RDF validation and querying standards such as SHACL and SPARQL are well established [32, 40]. For lightweight experimentation, JSON-LD Playground provides quick inspection of contexts and RDF output [41]. RDFLib and Redland provide low-level programmatic RDF processing [33, 34]. These tools are important ecosystem building blocks but are not, by themselves, unified authoring environments.

1.4.5 Comparison of Semantic Web Editing Tools

Table 1.1 compares a subset of Semantic Web editing tools across capabilities, including RDF authoring, ontology integration, source-to-RDF transformation, RDF storage/query, visualization, and AI assistance.

Table 1.1: Comparison of tools supporting RDF authoring and Semantic Web integration

Tool / Framework	RDF Authoring	Ontology Integration	Source-to-RDF Transform.	RDF Storage / Query	Visualization	AI Assistance
Protégé	✓	✓	–	–	✓	–
WebProtégé	✓	✓	–	–	Limited	–
Apache Jena	✓	✓	Limited	✓	–	–
LinkML	–	✓	Partial	–	–	–
OpenRefine (RDF)	Limited	–	✓	–	–	–
SPARQL-Generate	–	–	✓	–	–	–
JSON-LD Playground	Limited	–	Limited	–	Limited	–
RDFLib	✓	Limited	–	Partial	–	–
Redland RDF	✓	Limited	–	Partial	–	–

Table 1.2: Rating scale used in Table 1.1

Symbol / Term	**Meaning**
✓	Native, first-class support
Partial	Substantial but scope-limited support (often programmatic rather than end-user-focused)
Limited	Basic or indirect support (e.g., plugin-based, narrow subset, or lightweight/read-only capability)
–	Not a core capability

1.5 Research Gap and Positioning of This Thesis

Although these tools provide strong support for individual Semantic Web tasks, several limitations remain for practical data workflows:

- **Fragmented workflows:** Modeling, mapping, querying, and visualization are often distributed across multiple tools, requiring users to switch between different environments to complete a single data integration task [18].
- **High technical entry barrier:** Effective use often requires advanced knowledge of OWL, RDF, SPARQL, and mapping languages, which limits accessibility for domain scientists who are not Semantic Web experts [22].
- **Gap between structured JSON editing and RDF transformation:** In practice, much scientific data is authored and maintained in hierarchical formats such as JSON. While standards such as JSON-LD provide a bridge to linked data [36], moving from conventional JSON structures to RDF still often requires separate mapping artifacts and additional tools (e.g., R2RML/RML) [11, 12].
- **Limited end-to-end support in one user interface:** Most tools focus on a single task—such as ontology editing, mapping definition, or querying—rather than supporting the entire workflow from data authoring and transformation to validation, querying, and graph exploration in a unified environment.
- **Limited AI support in semantic authoring workflows:** Despite recent advances in generative AI, assistance for generating artifacts such as SPARQL queries or RML mappings from natural-language descriptions of user intent is still rarely integrated into Semantic Web authoring tools.

This thesis addresses these gaps by extending MetaConfigurator from a schema-guided data modeling tool into an integrated RDF authoring environment. In particular, it supports a continuous workflow from JSON authoring to JSON-LD/RDF transformation, ontology-aware triple editing, integrated SPARQL querying, graph visualization, and AI-assisted drafting of SPARQL queries and RML mappings.

2 Design and Implementation

This chapter explains how the Semantic Web extensions were implemented in MetaConfigurator. The implementation supports a continuous workflow from conventional structured data to semantically enriched JSON-LD/RDF, including triple authoring, SPARQL querying, and KG exploration in one interface. The chapter is structured along this workflow: Section 2.1 introduces the RDF Authoring Panel (including RML-based transformation, panel composition, linked selection, and ontology integration), Section 2.2 covers query authoring and execution with optional AI support, and Section 2.3 presents interactive KG visualization.

2.1 RDF Authoring Panel

The RDF authoring functionality is implemented as a dedicated panel in MetaConfigurator (`meta_configurator/src/components/panels/rdf`). This keeps semantic authoring modular while remaining integrated with existing panel configuration, path selection, and data editing flows [25]. At runtime, the entry component `RdfPanel.vue` checks whether the current data can be interpreted as JSON-LD and immediately reports issues such as missing `@context`, invalid syntax, or non-JSON-LD input.

More generally, panel integration in MetaConfigurator follows a decoupled architecture: panels do not update each other directly, but communicate through a central reactive store as the single source of truth. Each panel writes data updates to the store and watches store state for incoming changes.

Figure 2.1 shows a snapshot of the RDF panel in action, loaded with the JSON example from 1.1. Since the data is provided in JSON format, the panel indicates that it must first be converted to JSON-LD before further processing. The user can proceed in two ways: either by directly supplying a JSON-LD document in the Text View, or by using the JSON-to-JSON-LD conversion tool discussed in Section 2.1.1.

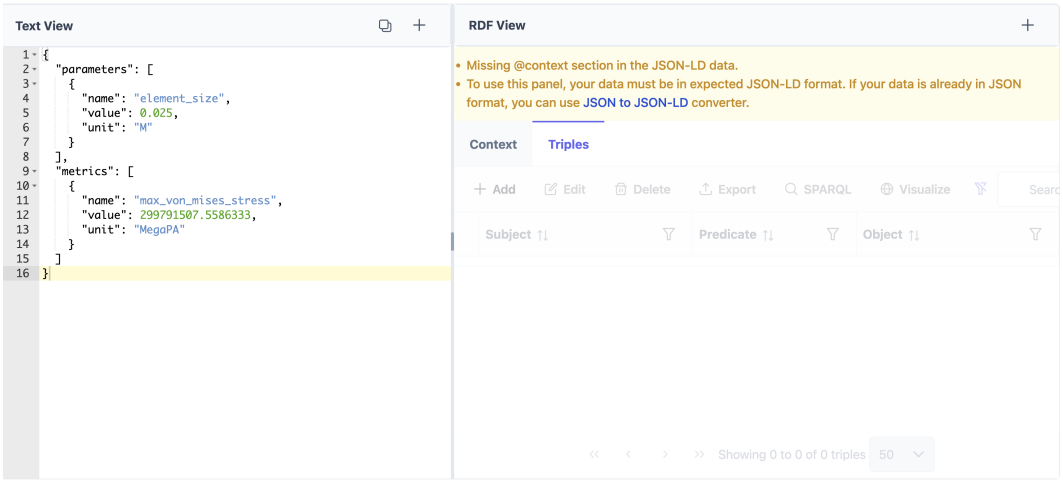

Figure 2.1: A snapshot of MetaConfigurator's RDF panel.

2.1.1 RML Mapping Dialog

If the input is not already available as JSON-LD, the RDF panel offers a conversion dialog that maps JSON data to RDF-compatible JSON-LD using RML. RML extends R2RML and supports transformation from heterogeneous sources such as JSON, CSV, and XML into RDF graphs [12] (see Section 1.2.7). Figure 2.2 shows the mapping dialog.

The dialog supports two entry points: users can paste an existing RML mapping or generate a draft with AI assistance. For AI-assisted generation, users provide short mapping hints that describe how elements of the input JSON should be mapped to RDF classes and properties. These hints are combined with a representative subset of the input data and sent to an external AI API. The generated prompt is structured to improve output quality, requiring Turtle syntax and RML-compliant constructs. In simplified form, it contains: (i) role and instruction constraints, (ii) required prefixes and output format, (iii) representative input fragments, and (iv) user-provided mapping hints. Listing 2.1 shows a shortened excerpt of this prompt logic.

The mapping editor is built on CodeMirror, which provides syntax highlighting and automatic indentation [17]. A debounced validation routine checks mapping syntax while users edit. This setup lets users inspect, refine, and validate a mapping before applying it to transform JSON into JSON-LD. Regardless of whether the mapping is pasted manually or generated via AI, it is presented in the same editor for final review and correction.

Since AI-generated mappings may contain semantic or syntactic errors, users must verify them before execution. Any syntax errors introduced during mapping editing, as well as errors that occur after applying the mapping, are reported to the user to support debugging and correction.

```
You are an assistant that generates RML mappings in Turtle syntax that convert JSON
    input into RDF.
Follow the RML specification strictly and avoid generating invalid RML constructs.
The output must contain ONLY valid Turtle mapping code and no explanations.

Logical source rules:
- Use rml:logicalSource
- Set rml:source to "Data.json"
- Use rml:referenceFormulation ql:JSONPath
- Use appropriate JSONPath iterators (e.g., "$.items[*]")

Object mapping / linking rules:
- In rr:objectMap, use exactly ONE of: rml:reference | rr:template | rr:constant
- If linking to another mapped resource, use rr:parentTriplesMap
- Do NOT combine rr:parentTriplesMap with rml:reference/rr:template/rr:constant
- For IRI objects from JSON values, use rr:template + rr:termType rr:IRI
- Do NOT use "$." inside rml:reference values

Prefix/output rules:
Use only provided prefixes (and user-provided ones), e.g.:
rr:  rml:  ql:  rdf:  xsd:
Declare prefixes at the top and return a single Turtle document.

Here is an example of the expected input and output:
...

Real input JSON subset:
  {user input}

Additional user comments:
  {mapping hints}

Final instruction: Return ONLY the RML mapping in valid Turtle syntax.
```

Listing 2.1: Part of the RML prompt template which guides the AI model

```
Convert my JSON Data to JSON-LD such that I have two nodes, each node representing a
    parameter or metric.
```

Listing 2.2: Sample RML hint which describes the mapping from JSON in 1.1 to a minimal JSON-LD in 2.4

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix ex: <http://example.com/ns#> .

<#ParameterMapping>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$.parameters[*]"
  ] ;

  rr:subjectMap [
    rr:template "http://example.com/parameter/{name}" ;
    rr:class ex:Parameter
  ] ;

  rr:predicateObjectMap [
    rr:predicate ex:parameterName ;
    rr:objectMap [ rml:reference "name" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:parameterValue ;
    rr:objectMap [ rml:reference "value" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:parameterUnit ;
    rr:objectMap [ rml:reference "unit" ]
  ] .

<#MetricMapping>
  rml:logicalSource [
    rml:source "Data.json" ;
    rml:referenceFormulation ql:JSONPath ;
    rml:iterator "$.metrics[*]"
  ] ;

  rr:subjectMap [
    rr:template "http://example.com/metric/{name}" ;
    rr:class ex:Metric
  ] ;

  rr:predicateObjectMap [
    rr:predicate ex:metricName ;
    rr:objectMap [ rml:reference "name" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:metricValue ;
    rr:objectMap [ rml:reference "value" ]
  ] ;
  rr:predicateObjectMap [
    rr:predicate ex:metricUnit ;
    rr:objectMap [ rml:reference "unit" ]
  ] .
```

Listing 2.3: RML mapping which converts JSON data in Listing 1.1 to JSON-LD in Listing 2.4

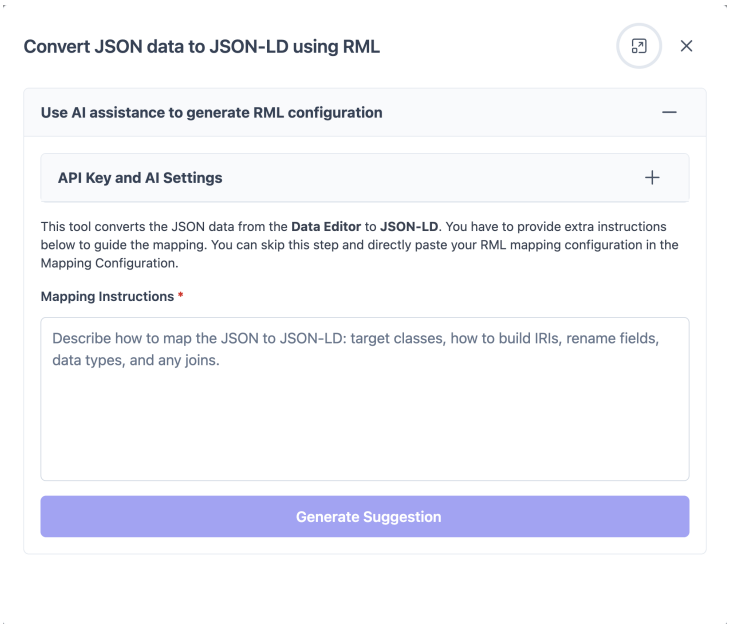

Figure 2.2: The RML mapping dialog for JSON-to-JSON-LD conversion.

Once finalized, the mapping can be applied to convert the input JSON data into JSON-LD, which can then be further managed in the RDF authoring panel.

For the JSON example presented earlier in Listing 1.1, a short natural-language instruction is sufficient to generate an executable RML draft that captures the intended structure (Listing 2.3). The resulting mapping already produces a consistent JSON-LD graph with stable resource identifiers, explicit types, and typed numeric values (Listing 2.4). This is important because it demonstrates the practical baseline of the workflow: low-effort hinting can work for straightforward mappings, while more complex semantics still require the detailed guidance.

In realistic scenarios, RML authoring is usually more complex and requires more explicit guidance to achieve the intended semantic structure. Detailed hints become especially important when the target output must follow specific vocabularies, IRI patterns, and inter-entity links. Listing 2.5 shows such a detailed hint. With this level of instruction, the generated mapping matches the expected structure in Listing 1.8 and produces the intended JSON-LD in Listing 1.4.

```
{
  "@context": {
    "ex": "http://example.com/ns#"
  },
  "@graph": [
    {
      "@id": "http://example.com/metric/max_von_mises_stress",
      "@type": "ex:Metric",
      "ex:metricName": "max_von_mises_stress",
      "ex:metricUnit": "MegaPA",
      "ex:metricValue": {
        "@type": "http://www.w3.org/2001/XMLSchema#double",
        "@value": "2.997915075586333E8"
      }
    },
    {
      "@id": "http://example.com/parameter/element_size",
      "@type": "ex:Parameter",
      "ex:parameterName": "element_size",
      "ex:parameterUnit": "m",
      "ex:parameterValue": {
        "@type": "http://www.w3.org/2001/XMLSchema#double",
        "@value": "2.5E-2"
      }
    }
  ]
}
```

Listing 2.4: JSON-LD representation of the simulation result, obtained after applying the minimal RML mapping hint shown above

```
Create one main resource local:run_fenics_simulation of type m4i:Method.
This node links to parameter nodes using m4i:hasParameter and to metric nodes using m4i:investigates.

Iterate over the arrays parameters[*] and metrics[*] separately to create parameter and metric nodes.

For each entry in these arrays:

Create a resource with IRI local:variable_{name} using the "name" field.
Set the type to schema:PropertyValue.
Set rdfs:label to the "name" value.
Set schema:value to the "value".
Map the "unit" field to schema:unitCode as an IRI from the QUDT unit vocabulary using the template
      http://qudt.org/vocab/unit/{unit}.

Connect the method node to parameter nodes using m4i:hasParameter, and to metric nodes using m4i:investigates.

Use these prefixes in the mapping:
m4i: http://w3id.org/nfdi4ing/metadata4ing#
schema: https://schema.org/
rdfs: http://www.w3.org/2000/01/rdf-schema#
unit: http://qudt.org/vocab/unit/
local: https://www.example.org/
```

Listing 2.5: Sample RML hints which describe the mapping from JSON in 1.1 to JSON-LD in 1.4

After users provide these hints, the configured AI model generates a mapping draft that aims to follow the requested structure and semantics. In practice, quality still depends on the underlying model, so correctness is not guaranteed. Once reviewed and confirmed by the user, the mapping can be executed to transform the input JSON into semantically enriched JSON-LD, as illustrated in Figures 2.3 and 2.4.

2.1.2 Panel Composition and UI Structure

The core editing surface is implemented in `RdfEditorPanel.vue` as a PrimeVue `Tabs`[4] container. The implementation is primarily organized into two tabs:

- **Context tab**: for editing `@context` definitions.
- **Triples tab**: for browsing and editing RDF triples.

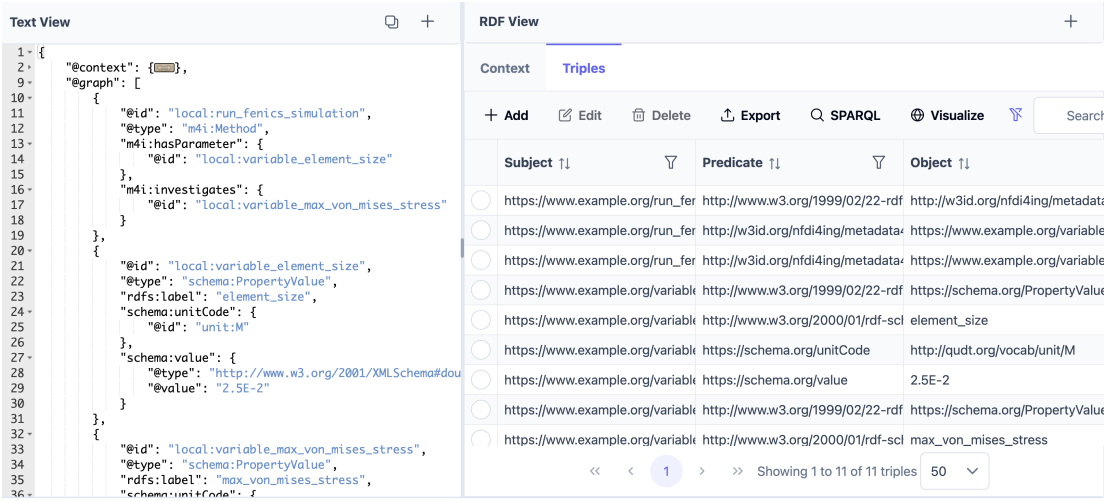

Figure 2.3: JSON-LD output after applying the mapping from Listing 1.8 in the Triples tab.

This explicit split mirrors the conceptual separation in JSON-LD between vocabulary declarations and graph assertions [36], and therefore supports both semantic modeling tasks (namespace/prefix handling) and graph authoring tasks (subject-predicate-object manipulation) in one workflow.

When the current document is invalid or not in the expected JSON-LD form, both tabs become visually disabled. This prevents accidental modifications when parsing preconditions are not satisfied.

[4] `https://primevue.org/tabs/`

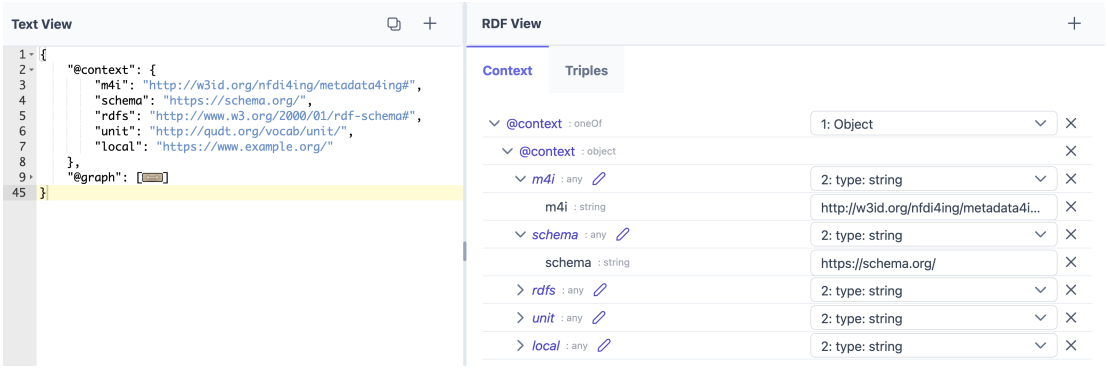

Figure 2.4: JSON-LD output after applying the mapping from Listing 1.8 in the Context tab.

2.1.3 Context Tab

The Context tab is implemented by `RdfContextEditorPanel.vue`. It reuses MetaConfigurator's GUI editor and applies a predefined JSON Schema so that `@context` can be edited through structured forms rather than plain text only. The selection updates are synchronized with the text editor view, enabling users to navigate to and manipulate context entries consistently with other panels (Figure 2.3).

2.1.4 Triples Tab

The Triples tab is implemented by `RdfTripleEditorPanel.vue`. It shows triples in a PrimeVue `DataTable`[5] with:

- paging, sorting, and column/global filtering,
- single-row selection synchronized with document path selection in the text editor,
- actions for add, edit, delete, export, SPARQL, and visualization.

Figure 2.4 shows the RDF panel focused on triples view. Triple editing is handled through `TripleDetailsDialog.vue`, which allows controlled entry of subject, predicate, and object terms, including typed literals with XML Schema datatypes. Changes are executed through `TripleEditorService` and propagated to the central RDF store. Figure 2.5 shows the edit modal for triple editing.

[5] `https://primevue.org/datatable/`

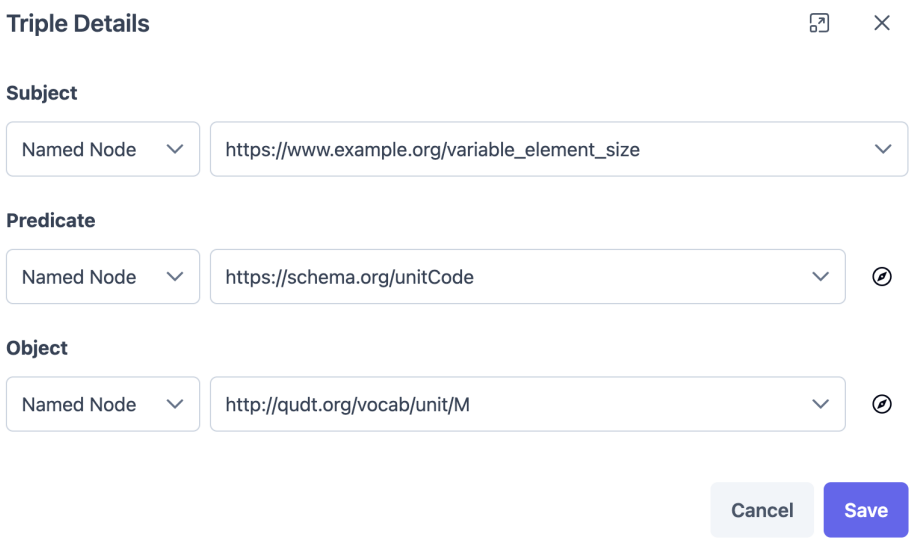

Figure 2.5: Edit modal for triple editing.

Data updates come from two places: the Text View (raw JSON-LD) and the RDF panel itself. Both paths write to MetaConfigurator's existing core data infrastructure, not to a new store introduced by this thesis. Specifically, `dataSource.ts` provides the global reactive source of editor data, which acts as the single source of truth across panels. On top of this, `managedData.ts` is the pre-existing synchronization layer that keeps parsed data and string representations aligned, supports path-based updates, and preserves temporarily unparseable text without discarding the last valid state. Within this architecture, the RDF extension plugs into the same flow: when a user edits the Text View, `rdfStoreManager.ts` rebuilds the RDF graph from the updated shared data state, which then updates the RDF panel (Figure 2.6).

Conversely, when a triple is added, edited, or removed in the Triples tab, `TripleEditorService` validates and applies the change through `rdfStoreManager.ts`.Then `jsonLdNodeManager.ts` rebuilds JSON-LD from the updated store and writes it back to `dataSource.ts`, which refreshes the Text View (Figure 2.7). Changes in the Context tab propagate through the same mechanism as GUI-based edits, because the underlying representation is still handled as JSON data by the same synchronization workflow.

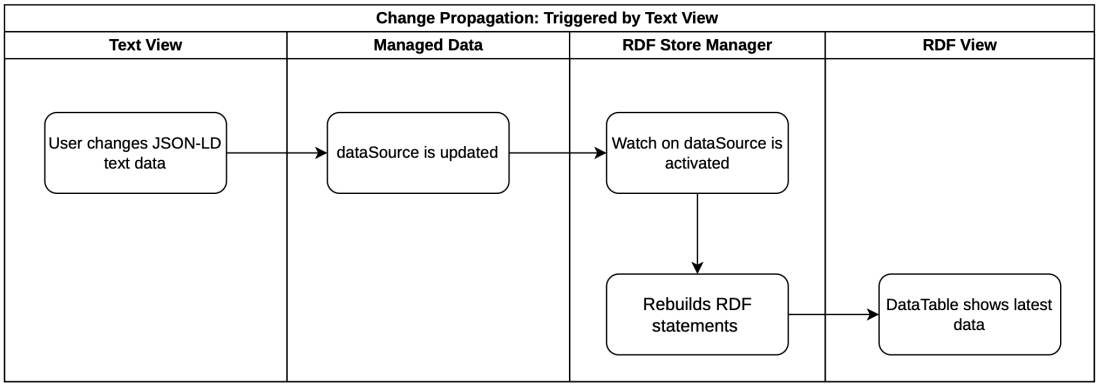

Figure 2.6: Synchronization process illustrating how modifications made in the Text View are propagated and reflected in the RDF View.

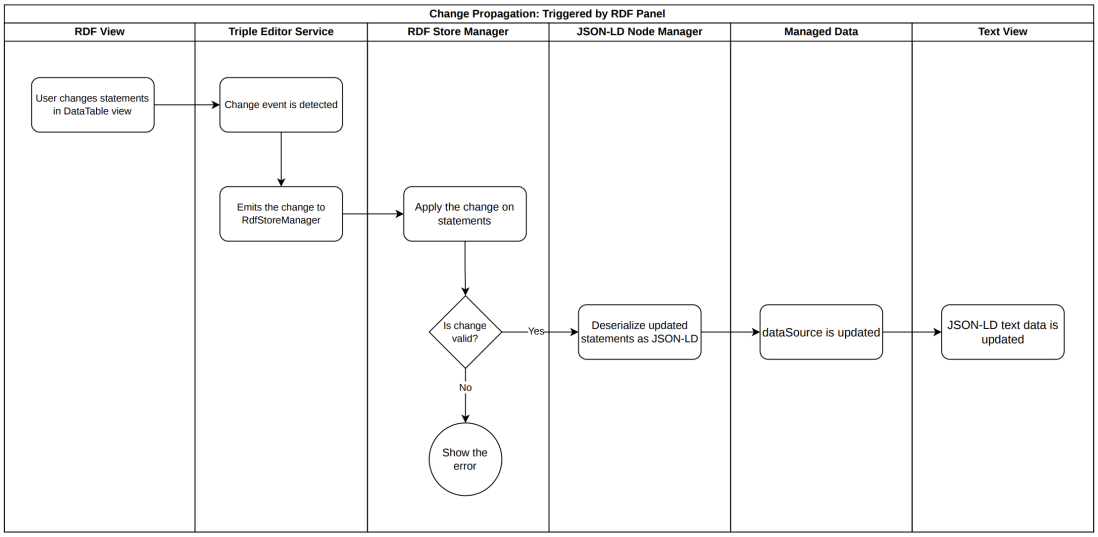

Figure 2.7: Workflow showing how changes originating in the RDF View are transferred back and represented in the Text View.

2.1.5 Ontology Integration and IRI Completion

Users can edit IRIs in the triple modal (Figure 2.5). For the `Predicate` and `Object` fields with the `Named Node` type, the panel provides a completion assistant by opening the Ontology Explorer dialog (`RdfOntologyExplorer.vue`). The explorer reads the JSON-LD `@context`, extracts prefix mappings, and presents them as selectable ontology scopes.

The interaction flow is shown in Figure 2.8. In this dialog, users can cache ontology content either by downloading it from a URL or by uploading a local file. The implementation detects supported RDF serializations (RDF/XML, Turtle, N-Triples, N3, JSON-LD), parses them in the browser using `rdflib.js`, serializes the resulting triples and stores both the original source content and the parsed graph data in IndexedDB[6].

Figure 2.9 illustrates how users can download the Schema.org ontology through the Ontology Explorer. After downloading an ontology or uploading a local file, the explorer updates a global cached graph by merging triples from all stored ontologies. This global graph is queried using the Comunica[7] library, allowing ontology terms to be explored consistently across previously cached sources. The predefined query retrieves `owl:ObjectProperty`, `owl:DatatypeProperty`, `rdf:Property`, `rdfs:Class`, and `owl:Class`. Property-related entries are displayed in both the `ObjectProperty` and `DatatypeProperty` tabs, while class definitions appear in the `Class` tab. Query results are cached per ontology prefix and recomputed only when the user explicitly triggers the `Refresh` action. To support large ontologies, each result table is paginated and includes table-level filtering.

Table 2.1 summarizes these vocabulary terms and their role in the explorer. In brief, the first three terms are property descriptors (for links between resources or from resources to literal values), while the last two denote class-level concepts used for type hierarchies in ontologies.

The predefined tabs work well for common schema elements (properties and classes), but they are limited to schema-level constructs. In real use cases, users also need instance-level resources and domain-specific concepts that do not fit these fixed categories. For example, in QUDT, a concrete unit such as `unit:M` (similar to Listing 1.4) is an instance of `qudt:Unit`. Such terms are not discoverable through class/property-only tabs.

For this reason, the explorer also provides a custom SPARQL interface (Figure 2.10). Users can run custom queries to locate classes, properties, and instances, with live syntax validation and inline error feedback. Listing 2.6 shows an example query that retrieves unit instances from QUDT.

[6] IndexedDB is a low-level, browser-based NoSQL database that enables web applications to store and efficiently query large amounts of structured data on the client side.

[7] `https://comunica.dev/`

Table 2.1: Vocabulary terms used for ontology term classification in the Ontology Explorer

Term	Description	Tab
`owl:ObjectProperty`	A property whose values are resources (IRIs or blank nodes), i.e., links between entities in the graph [42, 43].	`ObjectProperty`
`owl:DatatypeProperty`	A property whose values are literals such as strings, numbers, or dates [42, 43].	`DatatypeProperty`
`rdf:Property`	A generic RDF property class; in practice, used when no more specific OWL property typing is declared [6].	`ObjectProperty` and `DatatypeProperty`
`rdfs:Class`	The RDF Schema class of classes, used to define conceptual categories for resources [6].	`Class`
`owl:Class`	The OWL class construct used to model ontology concepts with richer semantics than plain RDF Schema typing [42, 43].	`Class`

Queries are executed against the cached global graph, and results are rendered with dynamically generated columns. If the results include IRI variables, the corresponding cells become selectable for IRI completion. This enables users to discover and select ontology terms that may not be easily accessible through the predefined views. Further details about the SPARQL component and its design are discussed in Section 2.2.

```
PREFIX qudt: <http://qudt.org/schema/qudt/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX dcterms: <http://purl.org/dc/terms/>

SELECT ?subject ?description WHERE {
  ?subject rdf:type qudt:Unit .

  OPTIONAL {
    ?subject dcterms:description ?description .
  }
}
```

Listing 2.6: Sample SPARQL query which returns instances of units defined in QUDT ontology

Figure 2.11 shows the result of the query in Listing 2.6. After execution, users can click an IRI-valued cell, inspect the resolved value in the selection bar, and confirm it with the `Select` button. The dialog then returns the chosen IRI to the triple modal, completing predicate or object entry. If an initial IRI already exists, the explorer automatically picks the best matching prefix, navigates to the relevant tab and page, and highlights the matching row for a consistent editing flow.

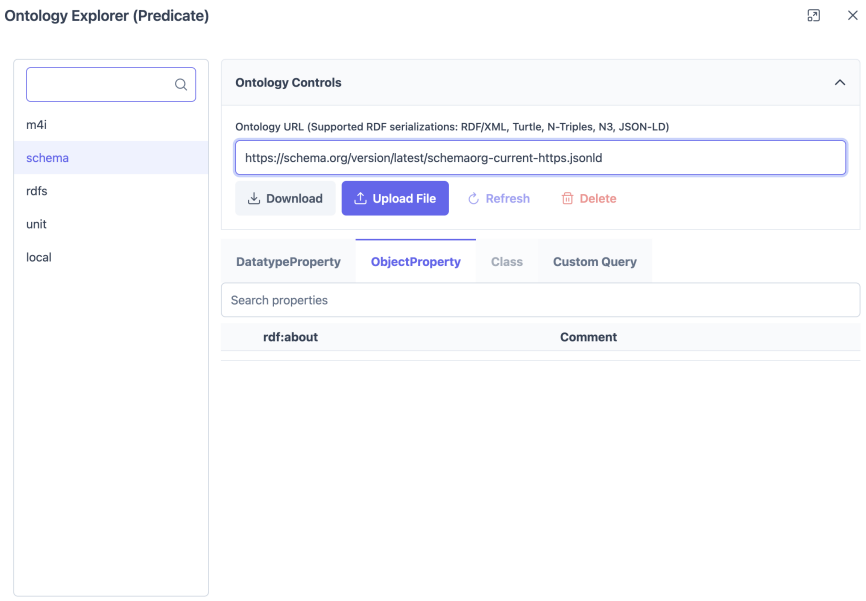

Figure 2.8: Ontology Explorer dialog, which allows users to download ontologies and browse their classes and properties for IRI completion.

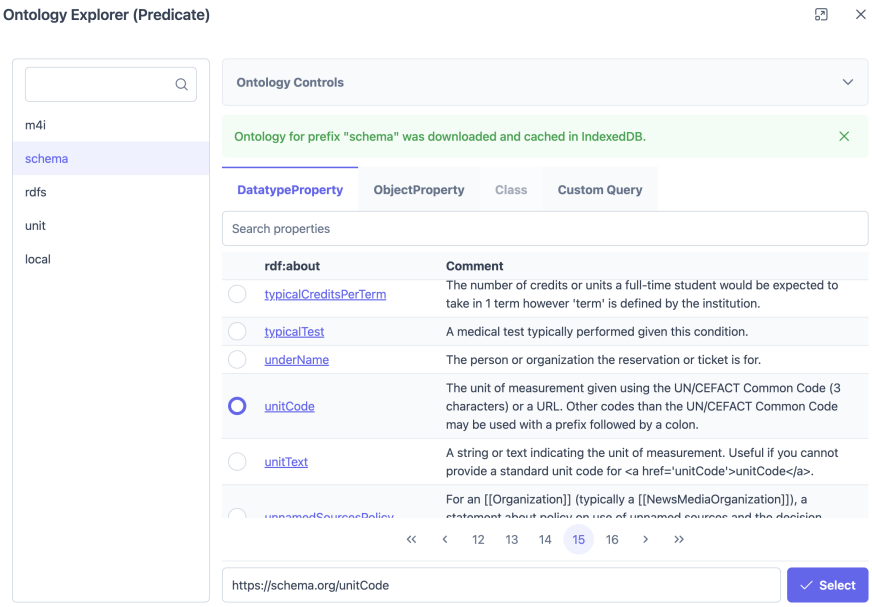

Figure 2.9: Downloading Schema.org ontology in the Ontology Explorer dialog.

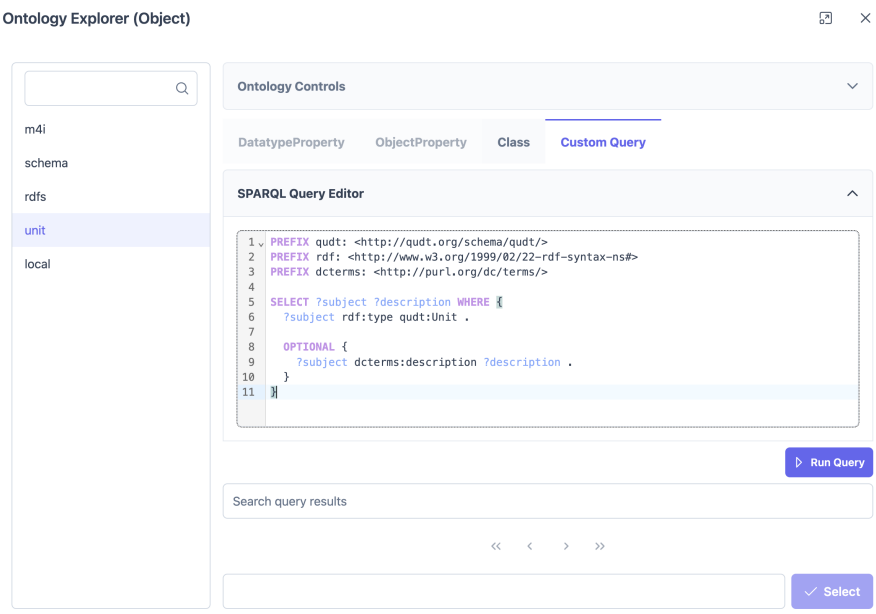

Figure 2.10: Ontology Explorer allows running custom SPARQL queries to find specific classes, properties or instances that are not accessible in predefined views.

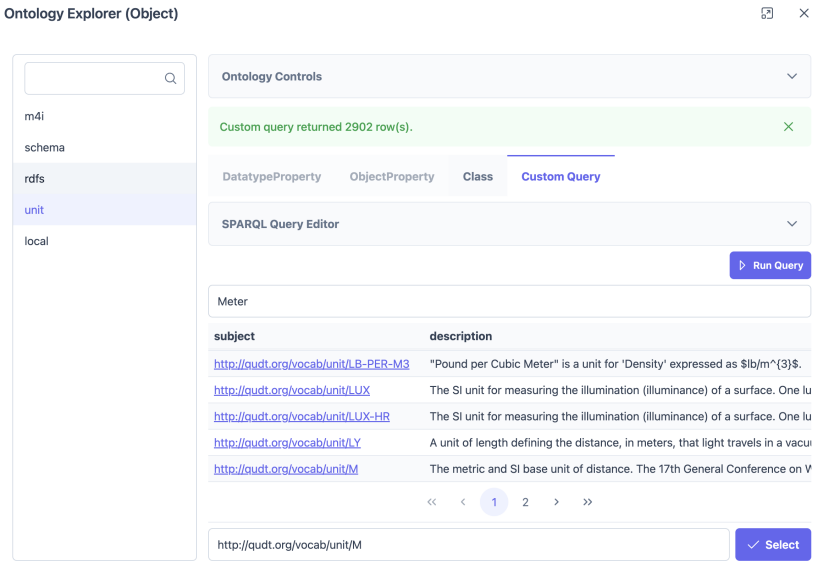

Figure 2.11: Results of running SPARQL query in Listing 2.6 over the QUDT ontology.

2.1.6 RDF Store and Data Synchronization

The internal semantic state is managed by `rdfStoreManager.ts`, which uses `rdflib.js` to parse and serialize data [23]. Because the synchronization logic operates on statement-level objects throughout this subsection, we first define the term *RDFLib Statement* used in the following description.

Definition 2.1 (RDFLib Statement)
In this implementation, an RDFLib Statement is the runtime representation of one RDF triple of the form (s, p, o)*, where* s *(subject),* p *(predicate), and* o *(object) are RDF terms. In* `rdflib.js`*, these terms are represented as* `NamedNode`*,* `BlankNode`*, and* `Literal` *objects, and each triple is stored as a Statement. Briefly, a* `NamedNode` *represents an identified resource via an IRI (e.g., ontology classes, predicates, or linked entities), a* `BlankNode` *represents an unnamed local resource without a global IRI, and a* `Literal` *represents a concrete value such as text, number, or date (optionally with datatype or language tag). A collection of such Statements forms the in-memory graph used by the panel [23].*

The `rdflib.js` library was selected because it provides mature browser-side handling of JSON-LD/RDF terms and integrates directly with statement-level editing operations used by the panel. On each source-data update, the manager:

1. resolves `@context` data into namespace mappings,
2. parses JSON-LD into an indexed RDF graph and derives a reactive list of Statements,
3. exposes reactive Statements, warnings, and errors to UI components.

The internal store parses JSON-LD into a reactive list of Statement objects, the triples table renders this list, and editing actions create or modify individual statements for add/edit/delete operations. Statement-level matching (subject, predicate, object) is used to synchronize table selection with text-editor paths.

To keep the JSON editor and RDF authoring panel consistent, the implementation maps RDF statements back into JSON-LD data and computes path correspondences between selected triples and document positions. This enables bidirectional synchronization: selecting a triple can focus the matching JSON path, and raw JSON data changes trigger store rebuilds.

2.1.7 Linked Selection Between Text View and RDF View

Linked selection between the Text View and the RDF View enables coordinated highlighting of corresponding elements across both representations. When a user selects a section of the JSON-LD document, the corresponding RDF statement is identified and highlighted in the triple table, and vice versa. This maintains a consistent visual selection state across both views. Figure 2.12 illustrates this behavior. Algorithms 2.1 and 2.2 describe the implementation of linked selection in both directions.

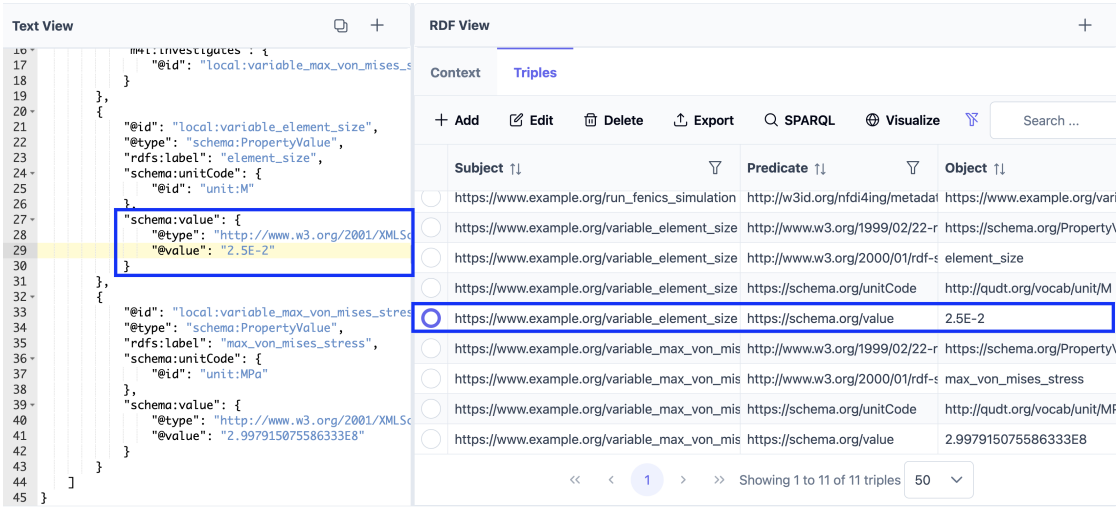

Figure 2.12: Linked selection between Text View and RDF View.

Algorithmus 2.1 Linked selection from Text View to RDF View

Input: current JSON path selection, full JSON-LD document, current triple table rows
Extract subject (`@id`), predicate, and object at the selected path in the JSON-LD document
Build a minimal JSON-LD fragment containing:
    a) original `@context`
    b) one `@graph` node with extracted subject, predicate, and object
Parse the minimal fragment with RDFLib into a temporary graph
if the temporary graph contains exactly one statement **then**
    Compare that statement against each triple table statement by subject, predicate, and object equality
    if only one matching statement is found **then**
        Determine its row index in the triple table
        Set triple table selection to the matching row
        Scroll/focus the table to the selected row

Algorithmus 2.2 Linked selection from RDF View to Text View

Input: selected triple table statement (s, p, o), current JSON-LD document
Read subject s, predicate p, and object o from the selected table row
Find candidate node in `@graph` whose `@id` matches s
if no candidate node is found **then**
    return without navigation
Find predicate entry in the candidate node that matches p
    (consider both prefixed terms and full IRIs as equivalent)
if no matching predicate entry is found **then**
    return without navigation
Match object o within the predicate value
    (as `@id`, `@value`, or scalar value)
if an object match is found **then**
    Compute the corresponding JSON path
    Move editor cursor/focus to that JSON path

2.2 Query with SPARQL

SPARQL querying is implemented in `SparqlEditor.vue` as a dialog with three tabs: `Query View`, `Result View`, and `Visualization`. RDFLib query support was initially evaluated, but it was too limited for the required in-browser workflow and did not fully cover current SPARQL 1.2 draft query features. The final implementation uses Comunica's in-browser `QueryEngine` [1], which supports SPARQL 1.2 draft query-related specifications according to official documentation [2].

In the **Query View**, users write and validate queries in a CodeMirror editor [17] and execute them against the in-memory RDF graph. Syntax checking is performed via debounced live validation using `Sparql.js`, which provides immediate feedback during editing [38]. The same view also supports AI-assisted query drafting. As in the RML workflow (Section 2.1.1), user hints are combined with prompt instructions and representative graph fragments, then sent to the configured AI endpoint. In simplified form, the prompt includes: (i) output constraints (SPARQL only), (ii) relevant prefixes and context, (iii) representative graph fragments, and (iv) the user hint. Figure 2.13 shows this interaction.

To demonstrate the AI-assisted workflow, we assume that the simulation JSON from Listing 1.1 has been extended with additional `element_size` and `max_von_mises_stress` value pairs (e.g., from multiple simulation runs), and then transformed into JSON-LD using the mapping workflow described earlier. In this extended dataset, metric measurements are available for multiple element sizes, for example `0.003125`, `0.00625`, `0.0125`, `0.05` and `0.1`.

In this scenario, the user provides a natural-language hint in the Query View to request a query that returns paired values of `element_size` and `max_von_mises_stress`. An example hint is shown in Listing 2.7. The hint and a subset of the current JSON-LD graph are sent to the AI API, which proposes a draft query that may still contain syntactic or semantic errors. Therefore, the generated query must be reviewed and, if necessary, refined by the user before execution. A representative generated query is shown in Listing 2.8.

```
Return pairs of element_size and max_von_mises_stress.
For each run, show the numeric element size value and the numeric stress value.
Sort by element_size ascending.
```

Listing 2.7: Example user hint for AI-assisted SPARQL generation

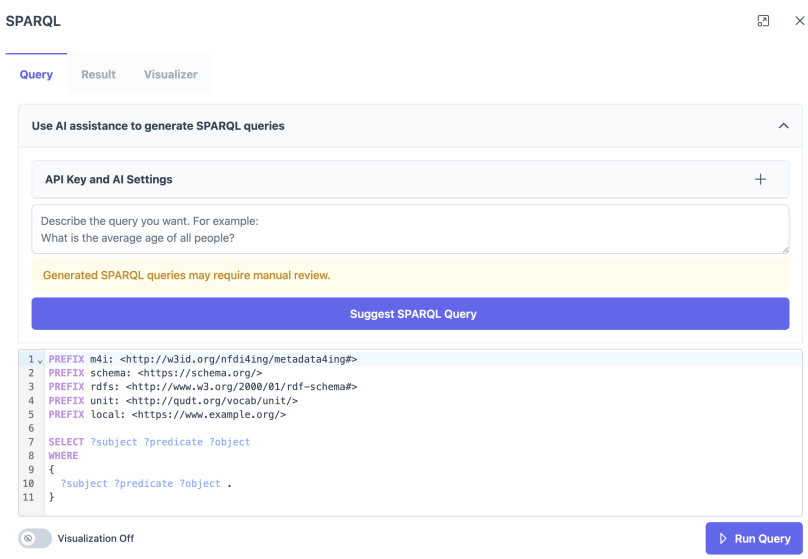

Figure 2.13: SPARQL query dialog, with AI-assisted query generation.

```
PREFIX m4i: <http://w3id.org/nfdi4ing/metadata4ing#>
PREFIX schema: <https://schema.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX unit: <http://qudt.org/vocab/unit/>
PREFIX local: <https://www.example.org/>

SELECT ?element_size ?max_von_mises_stress WHERE {
  ?method a m4i:Method .
  ?method m4i:hasParameter ?element_size_param .
  ?element_size_param a schema:PropertyValue ;
                      schema:value ?element_size .
  ?method m4i:investigates ?stress_param .
  ?stress_param a schema:PropertyValue ;
                schema:value ?max_von_mises_stress .
} ORDER BY ASC(?element_size)
```

Listing 2.8: AI-suggested SPARQL query for `element_size` vs. `max_von_mises_stress` pairs

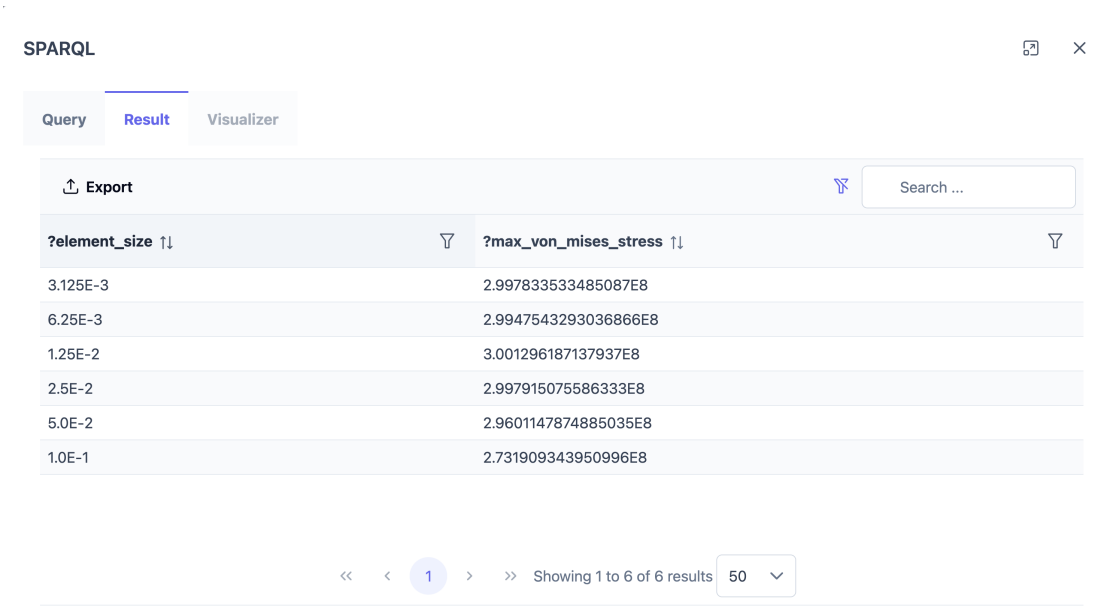

Figure 2.14: Result after running SPARQL query in Figure 2.13.

This showcase illustrates the intended interaction pattern: AI is used to accelerate query authoring, while final semantic and syntactic validation remains under user control before execution. Human-in-the-loop review is essential in both AI-generated RML mappings and AI-generated SPARQL queries, and users can inspect and refine both results before applying or executing them.

The dialog supports two execution modes controlled by a toggle: `Visualization On/Off`. With `Visualization Off`, standard SELECT-style querying is used and results are shown in the `Result` tab. In this tab, the PrimeVue table columns are generated dynamically from the returned variables, and export is provided as CSV. Figure 2.15 shows a plot which is generated from the CSV export of the query result, visualizing the relationship between `element_size` and `max_von_mises_stress`.

With `Visualization On`, the query mode switches to CONSTRUCT-based graph output; the same result table is still populated, but export options target RDF serializations (e.g., Turtle, N-Triples, RDF/XML).

The `Visualization` tab renders the query result graph in read-only mode. Its interaction model and rendering details are discussed in the Section 2.3.

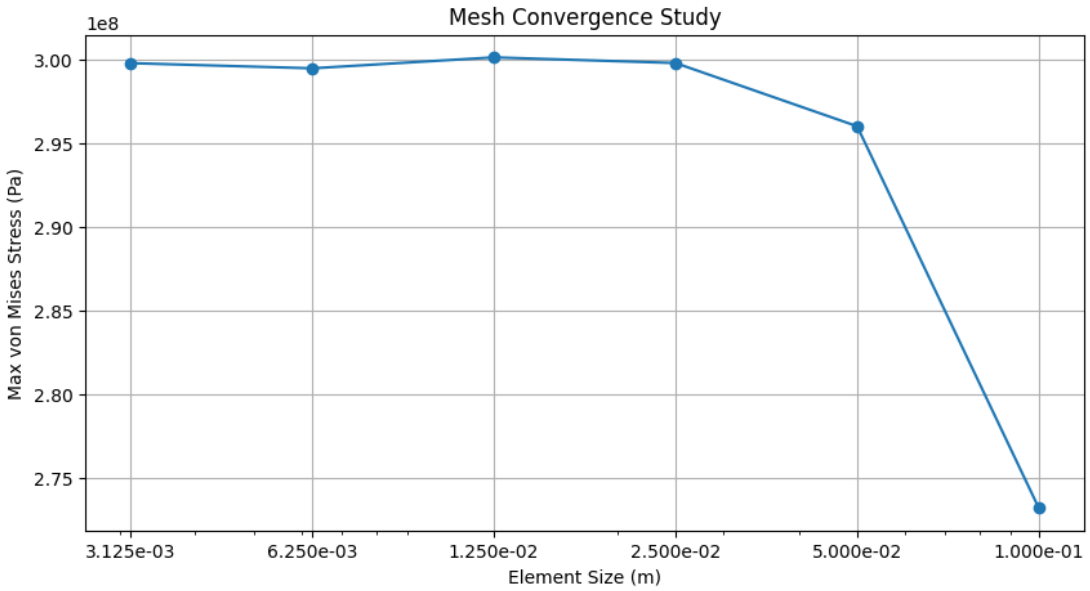

Figure 2.15: CSV export visualized: `element_size` vs. `max_von_mises_stress` trends.

2.3 Knowledge Graph Visualization

KG rendering is implemented in `RdfVisualizer.vue` using Cytoscape.js [14], with the COSE-Bilkent layout extension for force-directed graph placement. The visualization presents resources as nodes and semantic relations as directed edges, while preserving readable labels through namespace-aware prefix shortening.

The view provides interactive graph navigation features including zoom in/out, fit-to-view, and zoom-to-selected-node actions. For performance and usability, large graphs are detected before rendering and users are prompted to continue or cancel. In addition, graph export is supported as an image, and an optional animation mode can recompute the layout to improve readability after edits.

Node inspection and editing support are integrated through `RdfVisualizerPropertiesView.vue`. When a node is selected, a side panel shows the node identifier and its properties, including linkable IRIs. The same panel exposes authoring actions (add/edit/delete property and add/rename/delete node), so users can inspect and refine graph content directly from the visualization workflow.

The visualization tab can be opened in two ways. First, it can be opened directly from the Triples tab of the RDF panel. In this mode, the visualization is editable, and user interactions are propagated back to the main JSON-LD data and RDF store. Second, it can be opened from the SPARQL dialog when the `Visualization On` toggle is enabled. In that case, the graph is built from the user's SPARQL CONSTRUCT result and is shown in read-only mode, because returned triples are not guaranteed to map one-to-one to statements in the internal RDF store.

For editable visualization workflows, undo and redo actions are also integrated in the graph toolbar. This allows users to safely revise node/property edits while keeping visualization and underlying data synchronized.

To improve navigation in larger graphs, the visualization also provides node search via PrimeVue AutoComplete[8] component: users receive suggestions of available nodes while typing, can quickly select the intended node, and the view automatically focuses on it.

The visualization also supports clickable hyperlinks for semantic terms. Predicate labels can open their ontology definitions; for example, `m4i:hasParameter` links to predicate definition in the ontology[9]. For object values, `NamedNode` are handled context-sensitively: if the referenced node exists in the current graph, clicking it focuses that node in the visualization; if it points to an external resource, clicking opens the external definition (e.g., `unit:M` linking to `https://qudt.org/vocab/unit/M`).

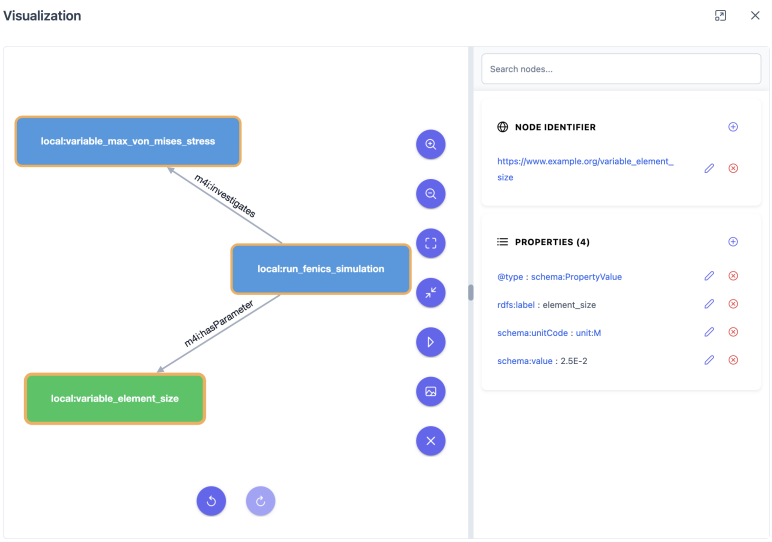

Figure 2.16: KG visualization of the JSON-LD from Listing 1.4 in the Visualization dialog.

[8] `https://primevue.org/autocomplete/`

[9] `https://nfdi4ing.pages.rwth-aachen.de/metadata4ing/metadata4ing/index.html#hasParameter`

3 Application in Chemistry

This chapter applies the full workflow of the thesis to Metal-Organic Framework (MOF) synthesis data. Structured laboratory records are modeled in JSON, transformed into semantically enriched RDF/JSON-LD with RML, and then used for ontology-aware querying and graph-based analysis.

Following the motivation in Chapters 1 and 2, the goal is to preserve protocol-level laboratory detail (materials, actions, and conditions) while adding formal semantics so the same data can be interpreted consistently across tools and datasets.

3.1 JSON Structure of the Chemical Dataset

MOF synthesis refers to the stepwise preparation of crystalline coordination materials from metal precursors and organic linkers under controlled laboratory conditions. Beyond the final product, the process is characterized by full protocol context, including reagents, roles, action sequence, and process conditions, which should be represented in structured, machine-readable form [27].

Listings 3.1 and 3.2 show a representative subset of this dataset. Each top-level entry under `Synthesis[*]` encodes one complete run (e.g., `S-1`, `S-2`) with four core components: hardware setup, run metadata, ordered procedure phases (Prep/Reaction/Workup), and reagent inventory with functional roles.

3.2 CHMO Ontology

The Chemical Methods Ontology (CHMO) provides a controlled vocabulary for chemical methods and related experimental concepts, including data-acquisition techniques, preparation and separation procedures, and synthesis methods [35]. Within the broader Open Biological and Biomedical Ontologies (OBO) ecosystem, CHMO is designed to complement other ontologies such as the Ontology for Biomedical Investigations (OBI), which captures more general investigation processes and experimental context [3, 21].

```
{
  "Synthesis": [
    {
      "Hardware": { "Component": [{ "_id": "S-1", "_type": "glass vial" }] },
      "Metadata": { "_description": "S-1", "_product": "MIL-88B (Fe)" },
      "Procedure": {
        "Prep": {
          "Step": [
            { "_vessel": "S-1", "$xml_type": "Add", "_amount": { "Unit": "MilliMOL", "Value": 0.4,
                 "$xml_append": "${Value} ${Unit}" }, "_reagent": "FeCl3", "_order": 0 },
            { "_vessel": "S-1", "$xml_type": "Add", "_amount": { "Unit": "MilliMOL", "Value": 0.4,
                 "$xml_append": "${Value} ${Unit}" }, "_reagent": "Benzene-1,4-dicarboxylic acid", "_order": 1
                 },
            { "_vessel": "S-1", "$xml_type": "Add", "_amount": { "Unit": "MilliL", "Value": 4, "$xml_append":
                 "${Value} ${Unit}" }, "_reagent": "DMF", "_order": 2 }
          ]
        },
        "Reaction": {
          "Step": [
            { "_vessel": "S-1", "$xml_type": "Sonicate", "_time": { "Value": 30.0, "Unit": "MIN",
                 "$xml_append": "${Value} ${Unit}" }, "_order": 0 },
            { "_comment": "In: oven", "_vessel": "S-1", "$xml_type": "HeatChill", "_temp": { "Unit": "DEG_C",
                 "Value": 120.0, "$xml_append": "${Value} ${Unit}" }, "_time": { "Value": 1, "Unit":"DAY",
                 "$xml_append": "${Value} ${Unit}" }, "_order": 1 }
          ]
        },
        "Workup": {
          "Step": [
            { "_vessel": "S-1", "$xml_type": "WashSolid", "_solvent": "EtOH", "_order": 0 },
            { "_vessel": "S-1", "$xml_type": "Dry", "_temp": { "Unit": "DEG_C", "Value": 120.0, "$xml_append":
                 "${Value} ${Unit}" }, "_time": { "Value": 1, "Unit":"DAY", "$xml_append": "${Value} ${Unit}"
                 }, "_pressure": { "Unit": "PA", "Value": 0, "$xml_append": "${Value} ${Unit}" }, "_order": 1 }
          ]
        }
      },
      "Reagents": {
        "Reagent": [
          { "_id": "FeCl3", "_name": "FeCl3", "_role": "substrate" },
          { "_id": "Benzene-1%252C4-dicarboxylic%2520acid", "_name": "Benzene-1,4-dicarboxylic acid", "_role":
               "ligand" },
          { "_id": "DMF", "_name": "DMF", "_role": "solvent" },
          { "_id": "EtOH", "_name": "EtOH", "_role": "solvent" }
        ]
      }
    }
  ]
}
```

Listing 3.1: Representative JSON excerpt from the MOF synthesis dataset for S-1

```
{
  "Synthesis": [
    {
      "Hardware": { "Component": [{ "_id": "S-2", "_type": "glass vial" }] },
      "Metadata": { "_description": "S-2", "_product": "MIL-88B+MIL101 (mix phases)" },
      "Procedure": {
        "Prep": {
          "Step": [
            { "_vessel": "S-2", "$xml_type": "Add", "_amount": { "Unit": "MilliMOL", "Value": 0.4,
                 "$xml_append": "${Value} ${Unit}" }, "_reagent": "FeCl3·6H2O", "_order": 0 },
            { "_vessel": "S-2", "$xml_type": "Add", "_amount": { "Unit": "MilliMOL", "Value": 0.4,
                 "$xml_append": "${Value} ${Unit}" }, "_reagent": "Benzene-1,4-dicarboxylic acid", "_order": 1
                 },
            { "_vessel": "S-2", "$xml_type": "Add", "_amount": { "Unit": "MilliL", "Value": 4, "$xml_append":
                 "${Value} ${Unit}" }, "_reagent": "DMF", "_order": 2 }
          ]
        },
        "Reaction": {
          "Step": [
            { "_vessel": "S-2", "$xml_type": "Sonicate", "_time": { "Value": 30.0, "Unit": "MIN",
                 "$xml_append": "${Value} ${Unit}" }, "_order": 0 },
            { "_comment": "In: oven", "_vessel": "S-2", "$xml_type": "HeatChill", "_temp": { "Unit": "DEG_C",
                 "Value": 120.0, "$xml_append": "${Value} ${Unit}" }, "_time": { "Value": 1, "Unit":"DAY",
                 "$xml_append": "${Value} ${Unit}" }, "_order": 1 }
          ]
        },
        "Workup": {
          "Step": [
            { "_vessel": "S-2", "$xml_type": "WashSolid", "_solvent": "EtOH", "_order": 0 },
            { "_vessel": "S-2", "$xml_type": "Dry", "_temp": { "Unit": "DEG_C", "Value": 120.0, "$xml_append":
                 "${Value} ${Unit}" }, "_time": { "Value": 1, "Unit":"DAY", "$xml_append": "${Value} ${Unit}"
                 }, "_pressure": { "Unit": "PA", "Value": 0, "$xml_append": "${Value} ${Unit}" }, "_order": 1 }
          ]
        }
      },
      "Reagents": {
        "Reagent": [
          { "_id": "FeCl3%25C2%25B76H2O", "_name": "FeCl3·6H2O", "_role": "substrate" },
          { "_id": "Benzene-1%252C4-dicarboxylic%2520acid", "_name": "Benzene-1,4-dicarboxylic acid", "_role":
               "ligand" },
          { "_id": "DMF", "_name": "DMF", "_role": "solvent" },
          { "_id": "EtOH", "_name": "EtOH", "_role": "solvent" }
        ]
      }
    }
  ]
}
```


Listing 3.2: Representative JSON excerpt from the MOF synthesis dataset for `S-2`

Using CHMO serves two purposes. First, it anchors synthesis resources to a domain-specific ontology rather than leaving them as generic data records. Second, it improves interoperability, because CHMO-typed entities can be combined with OBI- and Chemical Entities of Biological Interest (ChEBI)-based descriptions in a shared RDF graph.

3.3 RML Mapping for CHMO Integration

The RML mappings in Listings 3.3 and 3.4 transform the JSON structures from Listings 3.1 and 3.2 into typed RDF resources (see Section 1.2.7). In MetaConfigurator, this is the key transition from conventional JSON input to the RDF/JSON-LD authoring and query workflow.

Listings 3.3 and 3.4 illustrate how the chemistry-oriented ontology design is encoded in executable RML. The first listing defines the synthesis-level mapping context: `rml:iterator "$.Synthesis[*]"` selects each run in the input data, and `rr:subjectMap` creates one stable resource per run. Assigning `rr:class obo:COB_0000082` explicitly types each generated subject as an ontology-grounded synthesis process entity rather than a generic node.

The same mapping block also shows how experimental metadata is semantically attached to the synthesis resource. Fields such as vessel identifier, vessel type, human-readable run label, and product are transferred through `rr:predicateObjectMap`. This mapping pattern preserves the operational laboratory context while simultaneously lifting the data into an ontology-aware representation that can be interpreted by semantic tools.

The second listing extends this approach to reagents and preparation steps. Reagent resources are typed with `obo:CHEBI_24431`, aligning them with ChEBI-based chemical entities, while preparation-step resources are typed with `obo:OBI_0000070`, aligning procedural actions with OBI-based process semantics. The mapping therefore does more than copy JSON values into triples: it creates a typed graph where substances, methods, and process phases can be distinguished and queried systematically.

From a workflow perspective, this is crucial for the integration goals discussed in Chapters 1 and 2. Once the mapped graph is loaded in MetaConfigurator, users can formulate SPARQL queries over synthesis-level entities, reagent roles, and process steps without relying on path-specific JSON logic. The same typed structure also improves graph visualization readability, because nodes and edges represent explicit domain semantics rather than implicit document structure.

A central semantic effect of the mapping is ontology-based typing in `rr:class`. These class assignments transform plain JSON objects into explicit domain entities. For example, the mappings in Listings 3.3 and 3.4 assign the classes summarized in Table 3.1. In Listing 3.5, amount values are represented as typed nodes of type `schema:PropertyValue`, with the unit code linked to the QUDT ontology. Listing 3.6 shows part of the transformed data in JSON-LD format, representing synthesis run for `S-1`.

```
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix ex: <http://example.org/> .
@prefix obo: <http://purl.obolibrary.org/obo/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

<#SynthesisSource> rml:source "Data.json" ; rml:referenceFormulation ql:JSONPath ; rml:iterator "
      $.Synthesis[*]" .

<#SynthesisTriplesMap>
  rml:logicalSource <#SynthesisSource> ;
  rr:subjectMap [ rr:template "http://example.org/synthesis/{Hardware.Component[0]._id}" ; rr:class
        obo:COB_0000082 ] ;
  rr:predicateObjectMap [ rr:predicate schema:identifier ; rr:objectMap [ rml:reference "
        Hardware.Component[0]._id" ] ] ;
  rr:predicateObjectMap [ rr:predicate obo:RO_0000086 ; rr:objectMap [ rml:reference "
        Hardware.Component[0]._type" ] ] ;
  rr:predicateObjectMap [ rr:predicate rdfs:label ; rr:objectMap [ rml:reference "Metadata._description" ] ] ;
  rr:predicateObjectMap [ rr:predicate obo:OBI_0000299 ; rr:objectMap [ rml:reference "Metadata._product" ] ] .
```

Listing 3.3: Part of RML mapping focused on synthesis-level hardware and metadata

```
<#ReagentSource> rml:source "Data.json" ; rml:referenceFormulation ql:JSONPath ; rml:iterator "
      $.Synthesis[*].Reagents.Reagent[*]" .

<#ReagentTriplesMap>
  rml:logicalSource <#ReagentSource> ;
  rr:subjectMap [ rr:template "http://example.org/reagent/{_id}" ; rr:class obo:CHEBI_24431 ] ;
  rr:predicateObjectMap [ rr:predicate rdfs:label ; rr:objectMap [ rml:reference "_name" ] ] ;
  rr:predicateObjectMap [ rr:predicate obo:RO_0000087 ; rr:objectMap [ rml:reference "_role" ] ] .

<#PrepStepSource> rml:source "Data.json" ; rml:referenceFormulation ql:JSONPath ; rml:iterator "
      $.Synthesis[*].Procedure.Prep.Step[*]" .

<#PrepStepTriplesMap>
  rml:logicalSource <#PrepStepSource> ;
  rr:subjectMap [ rr:template "http://example.org/step/prep/{_vessel}-{_order}" ; rr:class obo:OBI_0000070 ] ;
  rr:predicateObjectMap [ rr:predicate obo:OBI_0000293 ; rr:objectMap [ rr:constant "Prep" ] ] ;
  rr:predicateObjectMap [ rr:predicate obo:RO_0000057 ; rr:objectMap [ rr:template "
        http://example.org/reagent/{_reagent}" ] ] ;
  rr:predicateObjectMap [ rr:predicate obo:RO_0000056 ; rr:objectMap [ rml:reference "$xml_type" ] ] ;
  rr:predicateObjectMap [
        rr:predicate obo:OBI_0001937 ;
        rr:objectMap [
            rr:template "http://example.org/quantity/prep/{_vessel}-{_order}/amount"
        ]
    ] .
```

Listing 3.4: Part of RML mapping focused on reagents and preparation steps

```
<#PrepAmountValueMap>
    rml:logicalSource <#PrepStepSource> ;

    rr:subjectMap [
        rr:template "http://example.org/quantity/prep/{_vessel}-{_order}/amount" ;
        rr:class schema:PropertyValue
    ] ;

    rr:predicateObjectMap [
        rr:predicate rdfs:label ;
        rr:objectMap [ rr:constant "amount" ]
    ] ;

    rr:predicateObjectMap [
        rr:predicate schema:value ;
        rr:objectMap [
            rml:reference "_amount.Value" ;
            rr:datatype xsd:double
        ]
    ] ;

    rr:predicateObjectMap [
        rr:predicate schema:unitCode ;
        rr:objectMap [
            rr:template "http://qudt.org/vocab/unit/{_amount.Unit}"
        ]
    ] .
```

Listing 3.5: Part of RML mapping focused on creating amount value nodes for preparation steps

Table 3.1: Ontology classes and properties used in `rr:class` and `rr:predicate` assignments in Listings 3.3 and 3.4

Class / Property IRI	**Definition in this mapping context**
Classes (`rr:class`)	
`obo:COB_0000082`	A process that is initiated by an agent.
`obo:CHEBI_24431`	Types each reagent resource as a chemical entity (e.g., ligand, solvent, substrate).
`obo:OBI_0000070`	Types each procedure-step resource as a planned process entity.
`schema:PropertyValue`	Types each quantity node encoding a numeric value and its unit.
Properties (`rr:predicate`)	
`schema:identifier`	Assigns the local alphanumeric identifier of the synthesis component (e.g., `S-1`).
`obo:RO_0000086`	Links the synthesis vessel to its physical container quality (e.g., glass vial).
`obo:BFO_0000051`	Relates a synthesis process to its constituent procedure steps (*has part*).
`obo:OBI_0000299`	Links a synthesis process to its specified output product (*has specified output*).
`obo:RO_0000056`	Relates a process step to the action type it realises (e.g., Add, Sonicate).
`obo:OBI_0001937`	Links a process step to its quantity value specification (*has value specification*).
`obo:OBI_0000293`	Indicates the procedural phase a step belongs to (*has specified input*).
`obo:RO_0000057`	Associates a process step with its participant reagent (*has participant*).
`obo:RO_0000087`	Assigns the chemical role of a reagent (e.g., substrate, ligand, solvent).
`schema:unitCode`	Specifies the unit of measurement for a quantity value node.
`schema:value`	Holds the numeric value within a quantity value node.

```
{
  "@context": {
    "ex": "http://example.org/",
    "obo": "http://purl.obolibrary.org/obo/",
    "rdfs": "http://www.w3.org/2000/01/rdf-schema#",
    "schema": "https://schema.org/",
    "xsd": "http://www.w3.org/2001/XMLSchema#",
    "qudt": "http://qudt.org/vocab/unit/"
  },
  "@graph": [
    {
      "@id": "ex:synthesis/S-1",
      "@type": "obo:COB_0000082",
      "schema:identifier": "S-1",
      "obo:RO_0000086": "glass vial",
      "obo:BFO_0000051": [ { "@id": "ex:step/prep/S-1-0" } ],
      "obo:OBI_0000299": "MIL-88B (Fe)",
      "rdfs:label": "S-1"
    },
    {
      "@id": "ex:step/prep/S-1-0",
      "@type": "obo:OBI_0000070",
      "obo:RO_0000056": "Add",
      "obo:OBI_0001937": { "@id": "ex:quantity/prep/S-1-0/amount" },
      "obo:OBI_0000293": "Prep",
      "obo:RO_0000057": { "@id": "ex:reagent/FeCl3" }
    },
    {
      "@id": "ex:quantity/prep/S-1-0/amount",
      "@type": "schema:PropertyValue",
      "rdfs:label": "amount",
      "schema:unitCode": { "@id": "qudt:MilliMOL" },
      "schema:value": { "@type": "xsd:double", "@value": "4.0E-1" }
    },
    {
      "@id": "ex:reagent/FeCl3",
      "@type": "obo:CHEBI_24431",
      "obo:RO_0000087": "substrate",
      "rdfs:label": "FeCl3"
    }
  ]
}
```

Listing 3.6: Representative JSON-LD excerpt from the MOF synthesis dataset for `S-1`

3.4 AI-Assisted SPARQL Generation

After applying the full mappings (partially depicted in Listings 3.3, 3.4 and 3.5), users can query the resulting RDF graph directly with SPARQL. In practical laboratory settings, however, writing correct SPARQL is often a bottleneck, especially for users with strong domain expertise but limited query-language experience. To reduce this friction, this chapter applies the same AI-assisted interaction pattern introduced in Chapter 2: users describe intent in natural language, and a configured AI endpoint proposes a first query draft.

For the MOF dataset, the main benefit is that users can start from experimental questions (e.g., "Which Prep steps used which reagent, and in what amount?") instead of manually constructing triple patterns from scratch. Listing 3.7 shows such a concise user hint. Based on this hint and a representative subset of the current JSON-LD/RDF graph, the AI-endpoint generates a candidate query, shown in Listing 3.8. Figure 3.1 shows the result of executing this query, which retrieves all preparation steps with their associated reagents and amounts for each synthesis run.

This workflow lowers the entry barrier for SPARQL authoring, reduces trial-and-error in the query editor, and lets domain experts focus more on interpretation than syntax details. At the same time, generated queries remain drafts: they may include modeling assumptions or predicate choices that need correction. A user-controlled review step is therefore still required before execution.

```
Retrieve all Prep steps with their reagents and amounts for each synthesis.
```

Listing 3.7: Example user hint for AI-assisted SPARQL generation

```
PREFIX ex: <http://example.org/>
PREFIX obo: <http://purl.obolibrary.org/obo/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX schema: <https://schema.org/>

SELECT ?synthesis ?step ?reagentLabel ?amount ?unit
WHERE {
  ?synthesis obo:BFO_0000051 ?step .

  ?step obo:OBI_0000293 "Prep" ;
        obo:RO_0000057 ?reagent ;
        obo:OBI_0001937 ?amountNode .

  ?amountNode schema:value ?amount ;
              schema:unitCode ?unit .

  ?reagent rdfs:label ?reagentLabel .
}
ORDER BY ?synthesis ?step
```

Listing 3.8: AI-suggested SPARQL query for Prep steps with reagent and amount information

Query | Result | Visualizer

Export

?reagentLabel	?amount	?unit	?synthesis	?step
FeCl3	4.0E-1	http://qudt.org/vocab/unit/MilliMOL	http://example.org/synthesis/S-1	http://example.org/step/prep/S-1-0
Benzene-1,4-dicarboxylic acid	4.0E-1	http://qudt.org/vocab/unit/MilliMOL	http://example.org/synthesis/S-1	http://example.org/step/prep/S-1-1
DMF	4.0E0	http://qudt.org/vocab/unit/MilliL	http://example.org/synthesis/S-1	http://example.org/step/prep/S-1-2
FeCl3·6H2O	4.0E-1	http://qudt.org/vocab/unit/MilliMOL	http://example.org/synthesis/S-2	http://example.org/step/prep/S-2-0
Benzene-1,4-dicarboxylic acid	4.0E-1	http://qudt.org/vocab/unit/MilliMOL	http://example.org/synthesis/S-2	http://example.org/step/prep/S-2-1
DMF	4.0E0	http://qudt.org/vocab/unit/MilliL	http://example.org/synthesis/S-2	http://example.org/step/prep/S-2-2

Figure 3.1: Result after running the SPARQL query from Listing 3.8.

3.5 Knowledge Graph Visualization and Editing

After converting synthesis records to JSON-LD, graph visualization becomes a direct step in the workflow. Users can open the Visualization view and inspect the same underlying RDF data as a KG, without additional transformation code. In this representation, synthesis runs, reagents, and process steps appear as connected nodes, while semantic relations are shown as labeled edges. Figure 3.2 illustrates this view for the mapped MOF data.

This immediate JSON-LD-to-KG transition is especially helpful for non-SPARQL users. Instead of interpreting nested JSON structures or triple tables, they can inspect protocol structure visually: which reagent belongs to which synthesis, how Prep/Reaction/Workup steps are linked, and where key metadata is attached. This also speeds up semantic consistency checks during exploratory analysis.

The same interface also supports lightweight editing of graph content. When a node is selected, the properties panel exposes its current predicates and values and allows users to add, edit, or delete properties directly. Node-level operations such as rename, add, and delete are also available. This means users can refine both structure and annotations in place, rather than switching between multiple tools.

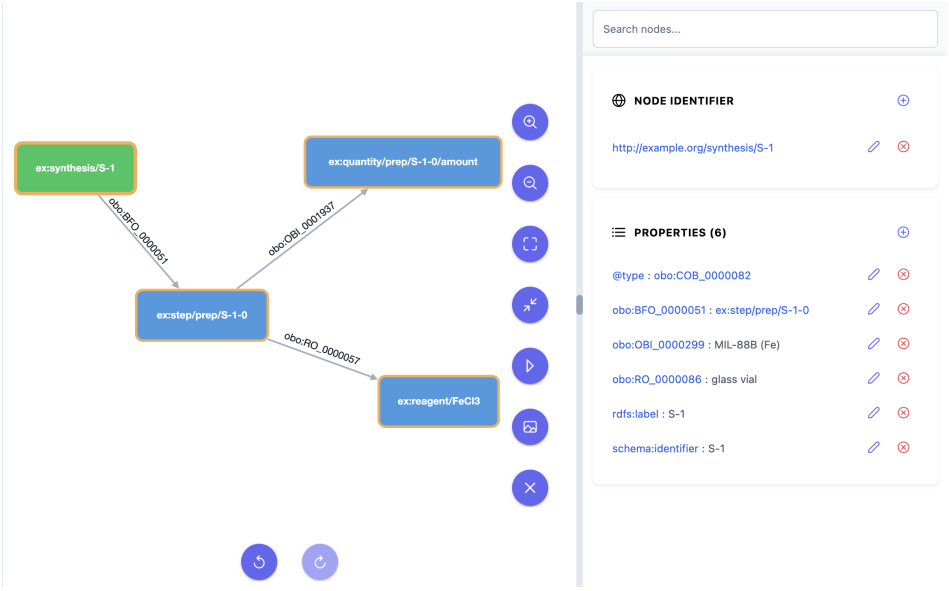

Figure 3.2: Visualization of the JSON-LD in Listing 3.6.

Crucially, these edits are synchronized with the underlying JSON-LD/RDF model. In other words, the KG is not only a static visualization layer; it is an interactive authoring surface. For end users, this reduces modeling effort and lowers the barrier to maintaining high-quality semantic data, because corrections can be performed where relationships are most intuitive: directly in the graph.

In addition, users can filter the graph semantically with SPARQL and visualize only the query result as a subgraph. This is particularly useful when the full dataset is large: instead of rendering all nodes, users can formulate a focused query (e.g., only Prep-related resources or only selected reagents), execute it, and inspect the returned structure directly in graph form. Hence, SPARQL-based filtering and KG visualization work together as a combined analysis workflow.

4 Conclusion and Outlook

This thesis extends MetaConfigurator with an integrated RDF authoring workflow that connects schema-driven JSON editing with Semantic Web technologies. The resulting workflow combines four capabilities in one interface: (1) transformation from JSON to JSON-LD via RML, (2) semantic editing through context and triple views, (3) in-browser SPARQL querying with validation support, and (4) interactive KG visualization with editing actions. AI-assisted generation of RML mappings and SPARQL queries further lowers the entry barrier for non-expert users, while keeping humans in control through explicit review before execution.

The application chapter demonstrates this workflow on MOF synthesis data. Laboratory protocol records are transformed into ontology-grounded RDF resources (CHMO, ChEBI, and OBI), then queried and explored as a graph. In practice, this shows that semantic data authoring can be carried out in one environment, reducing tool switching and improving interoperability.

4.1 Current Limitations

Despite these contributions, several limitations remain.

1. **Text-view serialization pattern after the first RDF edit:** After the first data edit in the RDF panel, the JSON-LD rendered in the Text View is regenerated from the internal RDF statements using a predefined serialization pattern. The resulting text can differ from the originally authored document even when the represented semantics stay equivalent. As shown in Listing 4.1, removing the triple:

   ```
   (local:variable_element_size , schema:name , "element_size")
   ```

   leads to additional textual changes such as renaming the `schema` prefix to `sch` and reordering literal fields (e.g., `@value` and `@type`). This behavior makes it harder to preserve original authoring style and document-level traceability.

2. **Remaining error risk in AI-assisted artifacts:** AI-generated mappings and queries can still contain semantic or structural errors. User review therefore remains essential, especially for domain-critical data and ontology alignment decisions.

3. **Scalability limits for browser-side workflows:** Scalability is still bounded by browser-side processing constraints for larger graphs, especially during interactive visualization and query-result rendering. To mitigate this, two configurable limits are already available:
   - `maximumTriplesToShow`: limits how many triples are rendered in the RDF panel to keep table rendering and interaction responsive.
   - `maximumNodesToVisualize`: limits how many nodes are rendered in the `Visualization` dialog to avoid unstable layout computation and slow graph interactions.

   Both settings can be adjusted according to dataset size and client performance.
4. **Ontology and IRI selection still requires expert knowledge:** Even with integrated mapping support, high-quality transformation from JSON to RDF still depends on choosing suitable ontologies, classes, predicates, and identifier patterns. Producing semantically robust mappings often requires domain and Semantic Web expertise (e.g., selecting appropriate ontology terms and stable IRIs), which remains a substantial barrier for non-expert users.

4.2 Outlook

Future work should focus on five directions:

1. **Preserve textual form during synchronization:** Future work should improve synchronization using path-aware or diff-based updates from statement changes back to the Text View. This is a syntactic and formatting concern: the semantic content is already preserved correctly, but the rendered textual form can drift from the originally authored style.
2. **Integrate semantic quality checks:** Semantic quality assurance should be strengthened through integrated SHACL validation in the authoring loop, so users can check constraints before export and publication.
3. **Improve scalability mechanisms:** Progressive graph loading, query-result size control, and graph summarization should be added to better support larger datasets.
4. **Support ontology discovery from current data context:** An additional extension would be an integrated recommendation feature that suggests suitable ontologies, classes, and predicates based on the currently loaded JSON data. This could reduce ontology-selection effort during mapping and help non-expert users choose semantically appropriate vocabularies.

Overall, the presented extension shows that user-centered semantic authoring in scientific workflows can be carried out within a single application, from structured JSON input to ontology-aware querying and KG analysis.

```
{
  "@context": {
    "m4i": "http://w3id.org/nfdi4ing/metadata4ing#",
    "unit": "http://qudt.org/vocab/unit/",
    "local": "https://www.example.org/",
    "sch": "https://schema.org/"
  },
  "@graph": [
    {
      "@id": "local:run_fenics_simulation",
      "@type": "m4i:Method",
      "m4i:hasParameter": {
        "@id": "local:variable_element_size"
      },
      "m4i:investigates": {
        "@id": "local:variable_max_von_mises_stress"
      }
    },
    {
      "@id": "local:variable_element_size",
      "@type": "sch:PropertyValue",
      "sch:unitCode": {
        "@id": "unit:M"
      },
      "sch:value": {
        "@value": "2.5E-2",
        "@type": "http://www.w3.org/2001/XMLSchema#double"
      }
    },
    {
      "@id": "local:variable_max_von_mises_stress",
      "@type": "sch:PropertyValue",
      "sch:name": "max_von_mises_stress",
      "sch:unitCode": {
        "@id": "unit:MegaPA"
      },
      "sch:value": {
        "@value": "2.997915075586333E8",
        "@type": "http://www.w3.org/2001/XMLSchema#double"
      }
    }
  ]
}
```

Listing 4.1: Deserilaized JSON-LD in Listing 1.1 after the first edit in the RDF Panel results in renaming `schema` prefix and reordering `@value` and `@type` fields

Bibliography

[1] C. Association. *@comunica/query-sparql*. 2026. URL: https://www.npmjs.com/package/@comunica/query-sparql (cit. on p. 54).

[2] C. Association. *Comunica: Supported specifications*. 2026. URL: https://comunica.dev/docs/query/advanced/specifications/ (cit. on p. 54).

[3] A. Bandrowski, R. Brinkman, M. Brochhausen, M. H. Brush, B. Bug, M. C. Chibucos, K. Clancy, M. Courtot, D. Derom, M. Dumontier, L. Fan, J. Fostel, G. Fragoso, A. Gonzalez-Beltran, M. A. Haendel, Y. He, M. Heiskanen, T. Hernandez-Boussard, M. Jensen, Y. Lin, A. Lister, J. Malone, E. Manduchi, M. McGee, J. A. Overton, H. Parkinson, B. Peters, P. Rocca-Serra, A. Ruttenberg, S.-A. Sansone, R. H. Scheuermann, D. Schober, B. Smith, L. N. Soldatova, C. J. Stoeckert, C. F. Taylor, C. Torniai, J. A. Turner, R. Vita, P. L. Whetzel, J. Zheng. "The Ontology for Biomedical Investigations". In: *PLOS ONE* 11.4 (2016), e0154556. DOI: 10.1371/journal.pone.0154556 (cit. on p. 59).

[4] T. Berners-Lee, J. Hendler, O. Lassila. "The Semantic Web". In: *Scientific American* 284.5 (2001), pp. 28–37 (cit. on pp. 15, 17).

[5] T. Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. 2017. URL: https://www.rfc-editor.org/rfc/rfc8259 (cit. on p. 16).

[6] D. Brickley, R. V. Guha. *RDF Schema 1.1*. 2014. URL: https://www.w3.org/TR/rdf-schema/ (cit. on p. 48).

[7] S. CBIR. *Protégé*. URL: https://protege.stanford.edu (cit. on p. 33).

[8] S.-G. CG. *SPARQL-Generate*. URL: https://github.com/SPARQL-Generate (cit. on p. 34).

[9] S. Cohen-Boulakia, K. Belhajjame, O. Collin, J. Chopard, C. Froidevaux, A. Gaignard, K. Hinsen, P. Larmande, Y. Le Bras, F. Lemoine, F. Mareuil, H. Ménager, C. Pradal, C. Blanchet. "Scientific workflows for computational reproducibility in the life sciences: Status, challenges and opportunities". In: *Future Generation Computer Systems* 75 (2017), pp. 284–298. DOI: 10.1016/j.future.2017.01.012 (cit. on p. 15).

[10] R. Cyganiak, D. Wood, M. Lanthaler. *RDF 1.1 Concepts and Abstract Syntax*. 2014. URL: https://www.w3.org/TR/rdf11-concepts/ (cit. on pp. 15, 19).

[11] S. Das, S. Sundara, R. Cyganiak. *R2RML: RDB to RDF Mapping Language*. https://www.w3.org/TR/r2rml/. W3C Recommendation. 2012 (cit. on p. 36).

[12] A. Dimou, M. Vander Sande, P. Colpaert, R. Verborgh, E. Mannens, R. Van de Walle. "RML: A Generic Language for Integrated RDF Mappings of Heterogeneous Data". In: *7th Workshop on Linked Data on the Web*. 2014 (cit. on pp. 26, 36, 38).

[13] A. S. Foundation. *Apache Jena*. URL: https://jena.apache.org/ (cit. on p. 33).

[14] M. Franz, C. T. Lopes, G. Huck, Y. Dong, O. Sumer, G. D. Bader. "Cytoscape.js: A graph theory library for visualisation and analysis". In: *Bioinformatics* 32.2 (2016), pp. 309–311. DOI: 10.1093/bioinformatics/btv557. URL: https://js.cytoscape.org/ (cit. on p. 57).

[15] R. S. Gonçalves, M. J. O'Connor, M. Martínez-Romero, et al. "The CEDAR Workbench: An Ontology-Assisted Environment for Authoring Metadata that Describe Scientific Experiments". In: *arXiv preprint arXiv:1905.06480* (2019) (cit. on p. 33).

[16] T. R. Gruber. "A Translation Approach to Portable Ontology Specifications". In: *Knowledge Acquisition* 5.2 (1993), pp. 199–220 (cit. on p. 17).

[17] M. Haverbeke. *CodeMirror: In-browser Code Editor*. 2023. URL: https://codemirror.net/ (cit. on pp. 38, 54).

[18] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. Labra Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, A. Zimmermann. "Knowledge Graphs". In: *ACM Computing Surveys* 54.4 (2021), pp. 1–37. DOI: 10.1145/3447772 (cit. on pp. 15, 33, 36).

[19] M. Horridge, R. S. Gonçalves, C. Nyulas, T. Tudorache, M. A. Musen. "WebProtégé: A Collaborative Web-Based Platform for Editing Biomedical Ontologies". In: *Bioinformatics* (2014) (cit. on p. 33).

[20] U. ISI. *Karma: A Data Integration Tool for Mapping Structured Data to RDF*. 2020 (cit. on p. 34).

[21] R. Jackson, N. Matentzoglu, J. A. Overton, R. Vita, J. P. Balhoff, P. L. Buttigieg, S. Carbon, M. Courtot, A. D. Diehl, D. M. Dooley, W. D. Duncan, N. L. Harris, M. A. Haendel, S. E. Lewis, D. A. Natale, D. Osumi-Sutherland, A. Ruttenberg, L. M. Schriml, B. Smith, C. J. Stoeckert, N. A. Vasilevsky, R. L. Walls, J. Zheng, C. J. Mungall, B. Peters. "OBO Foundry in 2021: operationalizing open data principles to evaluate ontologies". In: *Database* 2021 (2021), baab069. DOI: 10.1093/database/baab069 (cit. on p. 59).

[22] K. Janowicz, A. Haller, S. J. D. Cox, D. Le Phuoc, M. Lefrançois. "Why the Data Train Needs Semantic Rails". In: *AI Magazine* 36.1 (2015), pp. 5–14. DOI: 10.1609/aimag.v36i1.2568 (cit. on pp. 15, 33, 36).

[23] linkeddata. *rdflib.js Documentation*. 2026. URL: https://linkeddata.github.io/rdflib.js/doc/ (cit. on p. 51).

[24] S. Moxon, H. Solbrig, D. R. Unni, et al. "Linked Data Modeling Language (LinkML): A General-Purpose Data Modeling Framework". In: *Semantic Web J.* (2021) (cit. on p. 33).

[25] F. Neubauer, P. Bredl, M. Xu, K. Patel, J. Pleiss, B. Uekermann. “MetaConfigurator: A User-Friendly Tool for Editing Structured Data Files”. In: *Datenbank-Spektrum* 24 (2024), pp. 161–169. DOI: 10.1007/s13222-024-00472-7 (cit. on pp. 15, 28, 29, 33, 37).

[26] F. Neubauer, B. Uekermann, J. Pleiss. “AI-assisted JSON Schema Creation and Mapping”. In: *28th ACM/IEEE Int. Conf. Model Driven Eng. Languages and Systems Companion (MODELS-C)*. 2025, pp. 79–83. DOI: 10.1109/MODELS-C68889.2025.00019 (cit. on p. 30).

[27] F. Neubauer, K. Endo, F. Bender, E. Ciftci, N. Hansen, S. Krause, B. Uekermann, J. Pleiss, et al. “Data-Model-Driven Management and Analysis of MOF Synthesis Data”. In: (2026) (cit. on pp. 15, 59).

[28] F. Neubauer, J. Pleiss, B. Uekermann. “Data Model Creation with MetaConfigurator”. In: *Datenbanksysteme für Business, Technologie und Web (BTW 2025)*. Vol. P-361. Lecture Notes in Informatics (LNI), Proceedings. Gesellschaft für Informatik, Bonn, 2025. DOI: 10.18420/BTW2025-60. URL: https://dl.gi.de/handle/20.500.12116/45927 (cit. on pp. 28, 29, 33).

[29] NFDI4Ing Consortium. *Metadata4Ing Ontology*. 2022. URL: https://nfdi4ing.pages.rwth-aachen.de/metadata4ing/metadata4ing/ (cit. on p. 25).

[30] Ontotext. *GraphDB: RDF Database for Knowledge Graphs*. URL: https://www.ontotext.com/products/graphdb/ (cit. on p. 33).

[31] O. Project. *OpenRefine*. URL: https://openrefine.org/ (cit. on p. 34).

[32] E. Prud’hommeaux, A. Seaborne. *SPARQL 1.1 Overview*. 2013. URL: https://www.w3.org/TR/sparql11-overview/ (cit. on pp. 24, 34).

[33] RDFLib Project. *RDFLib: A Python library for working with RDF*. URL: https://rdflib.dev/ (cit. on p. 34).

[34] Redland RDF Project. *Redland RDF Libraries*. URL: http://librdf.org/ (cit. on p. 34).

[35] RSC Ontologies, OBO Foundry. *Chemical Methods Ontology (CHMO)*. Ontology resource and metadata at https://obofoundry.org/ontology/chmo.html. 2026. URL: http://purl.obolibrary.org/obo/chmo.owl (cit. on p. 59).

[36] M. Sporny, G. Kellogg, M. Lanthaler. *JSON-LD 1.0: A JSON-based Serialization for Linked Data*. 2014. URL: https://www.w3.org/TR/json-ld/ (cit. on pp. 21, 36, 43).

[37] J. F. Unger, A. Robens-Radermacher, S. M. Rosenbusch, D. Tyagi, D. Iglezakis, M. Jafarkhani. “A Platform for Validation and Verification of Models for Concrete and Concrete Structures”. In: *Computational Modelling of Concrete and Concrete Structures*. Taylor & Francis, 2026. Chap. A platform for validation and verification of models for concrete and concrete structures. DOI: 10.1201/9781003660026-32. URL: https://www.researchgate.net/publication/401659007 (cit. on p. 16).

[38] R. Verborgh, contributors. *SPARQL.js*. 2026. URL: https://www.npmjs.com/package/sparqljs (cit. on p. 54).

[39] J. Villamar, M. Kelbling, H. L. More, M. Denker, T. Tetzlaff, J. Senk, et al. “Metadata practices for simulation workflows”. In: *Scientific Data* 12 (2025), p. 942. DOI: 10.1038/s41597-025-05126-1 (cit. on p. 15).

[40] W3C. *Shapes Constraint Language (SHACL)*. 2017. URL: https://www.w3.org/TR/shacl/ (cit. on pp. 15, 21, 34).

[41] W3C JSON-LD Community Group. *JSON-LD Playground*. Online tool maintained by the W3C JSON-LD Community Group. 2023. URL: https://json-ld.org/playground/ (cit. on p. 34).

[42] W3C OWL Working Group. *OWL 2 Web Ontology Language Document Overview (Second Edition)*. 2012. URL: https://www.w3.org/TR/owl2-overview/ (cit. on pp. 25, 48).

[43] W3C OWL Working Group. *OWL 2 Web Ontology Language Primer (Second Edition)*. 2012. URL: https://www.w3.org/TR/owl2-primer/ (cit. on pp. 25, 48).

[44] S. R. Wilkinson, M. Aloqalaa, K. Belhajjame, M. R. Crusoe, B. de Paula Kinoshita, L. Gadelha, D. Garijo, O. J. R. Gustafsson, N. Juty, S. Kanwal, F. Z. Khan, J. Köster, K. Peters-von Gehlen, L. Pouchard, R. K. Rannow, S. Soiland-Reyes, N. Soranzo, S. Sufi, Z. Sun, B. Vilne, M. A. Wouters, D. Yuen, C. Goble. “Applying the FAIR Principles to computational workflows”. In: *Scientific Data* 12.1 (2025), p. 328. DOI: 10.1038/s41597-025-04451-9 (cit. on p. 15).

[45] A. Wright, H. Andrews, B. Hutton, G. Dennis. *JSON Schema: A Media Type for Describing JSON Documents*. IETF Draft. 2020. URL: https://json-schema.org/draft/2020-12/ (cit. on p. 16).

All links were last followed on April 06, 2026.

A Use of AI

During this thesis project, I used AI-based assistants, including OpenAI GPT-5.3-Codex[10] and Claude Sonnet 4.6[11], as support tools in software development. These tools were mainly used to speed up drafting and refinement of selected code segments related to the thesis implementation, for example by proposing initial structures, suggesting refactorings, and helping identify potential errors. AI-generated code was never integrated blindly: each suggestion was reviewed in context, tested, and revised where necessary to satisfy project requirements and maintainability goals.

I also used the same AI tools to support parts of the writing process for this document, such as improving phrasing, reorganizing draft text and generating preliminary adjustments for specific sections. All generated text was critically checked against the intended meaning, technical content, and academic standards. Final wording, structure, and claims were manually edited by me to ensure consistency, correctness, and alignment with the overall thesis. In addition, the "Principles for the Use of Artificial Intelligence (AI) by Students at the University of Stuttgart" were followed throughout the work.[12]

[10] OpenAI, "GPT-5.3-Codex," https://openai.com.

[11] Anthropic, "Claude Sonnet," https://www.anthropic.com/claude.

[12] University of Stuttgart, "Principles for the Use of Artificial Intelligence (AI) by Students at the University of Stuttgart,"
https://www.student.uni-stuttgart.de/digital-services/dokumente/ki-principles-for-students-en.pdf.

B Code and Data Availability

B.1 Data Availability

The JSON and JSON-LD data examined in Chapters 1 and 2 are publicly available on GitHub in the repository `NFDI4IngModelValidationPlatform`[13]. The repository contains workflows and documentation for the validation of constitutive models.

JSON data presented in the MOF synthesis study in Chapter 3 are also publicly available on GitHub in the repository `mof-synthesis-data-modeling`[14], with some modifications to comply with the intended RML mapping.

In addition, all data presented in this thesis are available in the GitHub repository `master-thesis-data`[15].

All repositories are distributed under the MIT License.

B.2 Code Availability

The source code of MetaConfigurator project is publicly available on GitHub in the repository `meta-configurator`[16].

The project is distributed under the MIT License.

[13] `https://github.com/BAMresearch/NFDI4IngModelValidationPlatform`
[14] `https://github.com/FAIRChemistry/mof-synthesis-data-modeling`
[15] `https://github.com/M-Jafarkhani/master-thesis-data`
[16] `https://github.com/MetaConfigurator/meta-configurator`

Declaration

I hereby declare that the work presented in this thesis is entirely my own and that I did not use any other sources and references than the listed ones. I have marked all direct or indirect statements from other sources contained therein as quotations. Neither this work nor significant parts of it were part of another examination procedure. I have not published this work in whole or in part before. The electronic copy is consistent with all submitted copies.

---

place, date, signature